ARTIFICIAL INTELLIGENCE IN MENTAL HEALTHCARE

A PROJECT REPORT

Submitted by:

MADHUMATHI S (6381344502 / 422623104003)
BALAJI P (9342636595 / 4226231035)
MALINI V (8807984385 / 4226231048)

BACHELOR OF ENGINEERING in COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING PANRUTI
PANRUTI-607106
ANNA UNIVERSITY, CHENNAI-600025
MAY 2026

ANNA UNIVERSITY: CHENNAI 600025
BONAFIDE CERTIFICATE

Certified that this project report "KEFFI AI – A MENTAL HEALTH CHATBOT" is the Bonafide work of MADHUMATHI S, BALAJI P, MALINI V who carried out their project under my supervision.

SIGNATURE

DR. S. SIVANESH M.Tech., Ph.D.
ASSISTANT PROFESSOR & HEAD OF DEPARTMENT
DEPARTMENT OF CSE
UNIVERSITY COLLEGE OF ENGINEERING PANRUTI

EXAMINED ON: 05/08/2026 Afternoon Session (AN) | Panel 2 | Chennai Region Venue

INTERNAL EXAMINER EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The successful completion of our project would not have been possible without the guidance, support, and encouragement of many people. We would like to express our sincere gratitude to all those who helped us throughout the completion of this project.

We express our heartfelt thanks to our respected Head of the Department and Project Guide Dr. S. Sivanesh, M.Tech., Ph.D., for his valuable guidance, continuous encouragement, and technical support throughout the project work. His suggestions and motivation helped us successfully complete our project.

We would also like to express our sincere thanks to our project coordinator Dr. K. Krishnakumari, M.E., Ph.D., for her support, guidance, and encouragement during every stage of the project development.

We extend our gratitude to all the teaching and non-teaching staff members of our department for their valuable suggestions and support.

Finally, we thank our parents and friends for their constant encouragement, motivation, and support throughout the completion of this project.

ABSTRACT

Keffi AI is a Clinical Digital Therapeutics Platform designed to improve digital mental healthcare by overcoming the limitations of existing chatbots, which lack memory, personalization, and clinical safety.

The system uses BERT-based emotion analysis to classify user input into 96 clinically relevant emotional states, enabling deeper understanding beyond basic sentiment detection. It

also incorporates a vector-based memory system
to retain past conversations and build a continuous psychological profile of the user.
A dynamic Mental Health Quotient (MHQ) tracks user condition in real time, while a predictive attrition algorithm
identifies early signs of disengagement and emotional decline. The platform integrates an automation layer (n8n) to
trigger real-time interventions such as alerts, reminders, and crisis responses.
Keffi AI provides both a patient interface for interaction and a clinical dashboard for monitoring, bridging the gap
between self-help tools and professional mental healthcare.

TABLE OF CONTENTS

CHAPTER TITLE PAGE NO

CHAPTER TITLE PAGE NO

LIST OF FIGURES

FIGURE NO FIGURE NAME PAGE NO

ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
NLP	Natural Language Processing
BERT	Bidirectional Encoder Representations from Transformers
RAG	Retrieval-Augmented Generation
MHQ	Mental Health Quotient
CBT	Cognitive Behavioral Therapy
ACT	Acceptance and Commitment Therapy

CHAPTER 1

INTRODUCTION

1.1 Overview of Mental Health

Mental health is an important part of human life that affects emotions, thinking ability, behavior, relationships, and daily activities. Problems such as depression, anxiety, stress, emotional burnout, and loneliness are increasing rapidly due to modern lifestyle changes, academic pressure, work stress, and social isolation. Many people suffer silently without proper emotional support or medical guidance.

Traditional mental healthcare systems mainly depend on therapist consultations and medication. Although these methods help many patients, they also have several limitations. Therapy sessions are expensive, time-consuming, and not easily accessible for everyone, especially people living in rural areas. Patients often receive support only during appointments, while their emotional condition between sessions remains unnoticed. This creates a gap in continuous mental

health monitoring.

1.2 Need for AI in Mental Healthcare

Artificial Intelligence (AI) and Natural Language Processing (NLP) technologies have created new opportunities in the healthcare field. AI systems can understand human language, analyze emotions, identify behavior patterns, and provide real-time responses. In mental healthcare, AI can help patients by providing continuous emotional support anytime and anywhere.

Existing chatbot systems such as Woebot and Wysa provide basic emotional conversations, but they mostly depend on predefined replies or generic AI-generated responses. These systems do not deeply understand user emotions, cannot remember past emotional conversations, and lack personalized therapeutic support. They also have limited crisis handling capabilities.

Therefore, there is a need for an intelligent mental healthcare platform that can:

- Understand emotions accurately
- Remember past conversations
- Provide personalized therapeutic responses
- Monitor mental health continuously
- Detect emotional risks early
- Support users during crisis situations

1.3 Proposed System – Keffi AI

To overcome the limitations of existing systems, the proposed project "Keffi AI" is developed as an AI-powered clinical mental healthcare assistant. The system combines Artificial Intelligence, Machine Learning, NLP, and therapeutic frameworks to provide emotionally aware and personalized mental health support.

Keffi AI uses a BERT-based emotion analysis model to classify user

emotions into 96 clinically relevant emotional states. Instead of detecting only basic emotions like happy or sad, the system can identify deeper emotional conditions such as hopelessness, anxiety, emotional burnout, treatment dropout risk, and depressive behavior. The system also uses Retrieval-Augmented Generation (RAG) and Pinecone vector memory to remember previous conversations and build a continuous psychological profile of the user. This allows the chatbot to provide context-aware and personalized responses rather than generic replies.

Another important feature is the Mental Health Quotient (MHQ) scoring system, which continuously tracks the emotional condition of the user and predicts emotional decline or risk conditions in real time.

1.4 Therapeutic Support System

Keffi AI provides scientifically validated therapeutic response strategies based on psychological frameworks such as:

• Cognitive Behavioral Therapy (CBT)

• Acceptance and Commitment Therapy (ACT)

• Grounding techniques

• Reflective questioning

• Empathy-based communication

Depending on the user’s emotional condition, the system selects the most appropriate therapeutic response method. The platform also includes crisis intervention protocols to handle emergency situations safely.

1.5 Objective of the Project

The main objective of Keffi AI is to create an intelligent and scalable digital mental healthcare platform that can provide:

• Continuous emotional monitoring

• Personalized mental health support

• Early risk detection

• Attrition prediction

• Crisis-safe intervention

• Improved accessibility to mental healthcare

The system aims to bridge the gap between traditional therapy systems and modern AI-driven healthcare technologies.

1.6 Hands-Free Real-Time Voice-to-Voice AI Architecture

Keffi AI integrates hands-free real-time Voice-to-Voice AI interaction via Web Speech API (STT & TTS). When a user speaks into the microphone, Keffi AI transcribes the utterance, analyzes emotional context, and speaks back Keffi's therapeutic reply in a soothing female voice (pitch = 1.05, rate = 0.95).

CHAPTER 2

LITERATURE SURVEY

2.1 Emotion Detection using Deep Learning

Author(s): A. Kumar, R. Singh

Year: 2021

This research focuses on detecting human emotions using Deep Learning techniques. The authors used a BERT-based emotion classification model trained on the GoEmotions dataset to identify multiple emotional states from textual conversations. Unlike traditional sentiment analysis methods that only classify text as positive or negative, the proposed model can understand complex emotions such as sadness, anxiety, anger, fear, and hopelessness. The study proved that transformer-based models like BERT provide better contextual understanding because they process text bidirectionally. This helps the system analyze emotional meaning more accurately. The research concluded that deep learning improves the accuracy of multi-emotion detection and is highly suitable for

mental healthcare applications.

This research supports the proposed Keffi AI system because it also uses a BERT-based model for fine-grained emotion analysis and personalized mental health support.

2.2 Sentiment Analysis for Mental Health Monitoring

Author(s): J. Smith et al.

Year: 2020

This research explains how Natural Language Processing and machine learning techniques can be used for monitoring mental health through sentiment analysis. The study used Twitter datasets to analyze emotional behavior and psychological conditions from social media conversations.

The authors applied NLP-based sentiment classification methods to detect emotional states such as sadness, stress, loneliness, and frustration. The study showed that social media conversations contain emotional patterns that can help identify mental health risks at an early stage.

The research concluded that sentiment analysis is an effective method for continuous emotional monitoring and psychological assessment.

This work is useful for the proposed Keffi AI system because it also performs real-time emotional monitoring using NLP techniques.

2.3 AI Chatbots for Mental Health Support

Author(s): L. Johnson

Year: 2019

This research focuses on the development of AI chatbot systems for providing mental health support. The proposed chatbot combined rule-based systems with conversational AI techniques to provide emotional assistance through text conversations.

The chatbot was trained using custom conversational datasets

containing therapeutic dialogues and emotional support patterns. The system provided immediate responses, motivational interaction, and basic self-help guidance for users experiencing stress and anxiety.

The research concluded that AI chatbots improve accessibility to mental healthcare by providing instant emotional support anytime and anywhere. However, the study also identified limitations such as lack of emotional memory and personalized interaction.

This research is important for the proposed Keffi AI platform because the system improves these limitations by integrating emotional memory, therapeutic AI responses, and crisis intervention mechanisms.

2.4 Detecting Depression from Text

Author(s): M. Sharma et al.

Year: 2022

This research focuses on identifying depression symptoms through textual conversation analysis using LSTM and NLP techniques. The study used the Reddit Mental Health Dataset to analyze emotional language patterns and depressive behavior.

The researchers found that individuals suffering from depression often use language associated with hopelessness, emotional withdrawal, loneliness, and negative thinking. The LSTM model effectively analyzed sequential text patterns and detected depressive tendencies with high accuracy.

The research concluded that NLP-based text analysis can help in early depression detection and psychological risk prediction. This study supports the proposed Keffi AI system because it also analyzes user conversations to identify depressive emotional states and provide early intervention.

2.5 Multi-Emotion Classification using Transformers

Author(s): P. Verma

Year: 2023

This research explains the use of transformer-based models such as BERT and RoBERTa for multi-emotion classification tasks. The study used the GoEmotions dataset to train advanced NLP models capable of identifying multiple emotional categories from textual conversations.

Unlike traditional machine learning algorithms, transformer models provide better contextual understanding and semantic analysis. This improves the accuracy of emotion recognition in complex emotional situations.

The research concluded that transformer architectures outperform traditional machine learning models in emotion detection tasks because of their deep contextual understanding capabilities.

This research is highly relevant to the proposed Keffi AI system because the platform also uses transformer-based emotion analysis for identifying clinically relevant emotional states and generating personalized therapeutic responses.

2.6 Multi-LLM 70B Engine Cascade & High Availability

Keffi AI implements a 70B Multi-LLM Cascade Engine utilizing Groq Llama-3.3-70B for sub-500ms response generation, with automatic failover to OpenAI ChatGPT-4o-mini API for 100% clinical availability.

2.7 SHAP & LIME Explainable AI (XAI) in Clinical Decision Support

The platform incorporates SHAP and LIME via `/api/explain_clinical_decision` to provide feature attribution weights, explaining why specific clinical interventions or risk ratings were assigned.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 Hardware Requirements

The proposed system requires basic hardware components for smooth execution of AI models, chatbot interaction, and database processing.

• Processor: Intel i3 / AMD equivalent or higher

• RAM: Minimum 4 GB

• Storage: 10 GB free disk space

• Device: Laptop / Desktop / Smartphone

• Internet: Stable internet connection

These hardware components support real-time emotion analysis and mental health monitoring.

3.2 Software Requirements

The proposed system is developed using modern AI and web technologies for efficient performance.

Operating System: Windows / Linux / macOS

Programming Languages: Python, JavaScript

Frontend: React.js, HTML, CSS, Tailwind CSS

Backend: Fast API / Flask

AI & NLP Libraries: TensorFlow / PyTorch, Transformers (Hugging Face), NLTK / SpaCy

Database: PostgreSQL, Redis, Pinecone

Development Tools: Google Colab, VS Code, Jupyter Notebook

These software tools help in emotion detection, chatbot response generation, and real-time communication.

CHAPTER 4

ANALYSIS OF PROJECT

4.1 Use Case Analysis for Data Preparation

The data preparation phase is one of the most important stages in the proposed Keffi AI system. In this phase, emotional and conversational datasets are collected, cleaned, processed, and prepared for training the AI model. The system mainly uses datasets such as GoEmotions and Clinical Dialogue datasets for emotion classification and therapeutic response generation.

Initially, raw text data is collected from emotional conversation datasets. The collected data may contain unwanted symbols, duplicate values, missing information, and irrelevant text. Therefore, preprocessing techniques such as tokenization, stop-word removal, lowercasing, and text normalization are performed.

After preprocessing, the data is labeled into multiple emotional categories such as sadness, anxiety, stress, hopelessness, emotional burnout, and depression-related states. The prepared data is then converted into embeddings using NLP techniques and stored for model training.

This phase helps improve the accuracy and performance of the BERT-based emotion detection system.

4.2 Use Case Analysis for Model Development Phase

The model development phase focuses on building the intelligent mental healthcare system using AI and NLP technologies. The prepared datasets are used to train the BERT-based transformer model for multi-emotion classification.

The system analyzes user conversations, detects emotional states, calculates MHQ scores, retrieves emotional memory from Pinecone, and generates personalized therapeutic responses. The model also performs risk analysis and crisis detection for high-risk emotional conditions.

The developed AI system integrates FastAPI backend services, vector memory systems, therapeutic response modules, and automation workflows to provide continuous emotional support and real-time mental healthcare assistance.

4.3 Use Case Analysis for Voice Prosody Acoustic Tracking

Keffi AI includes a Librosa-based Voice Prosody Analyzer (`voice_prosody_analyzer.py`) that extracts Fundamental Pitch (F0), RMS Energy, and Speech Rate (WPM) as acoustic biomarkers for clinical depression and vocal fatigue.

CHAPTER 5

SYSTEM DESIGN

5.1 System Design Overview

The proposed system “Keffi AI” is designed as an intelligent AI-powered mental healthcare platform that combines emotion detection, therapeutic response generation, memory systems, and real-time communication. The system architecture is designed using multiple interconnected modules to ensure continuous emotional monitoring, personalized support, and crisis-safe intervention.

The system follows a layered architecture consisting of:

• User Interface Layer

• Backend Processing Layer

• AI & NLP Layer

• Memory Layer

• Database Layer

• Automation & Communication Layer

Each layer performs a specific function and communicates with other modules to provide intelligent mental healthcare support.

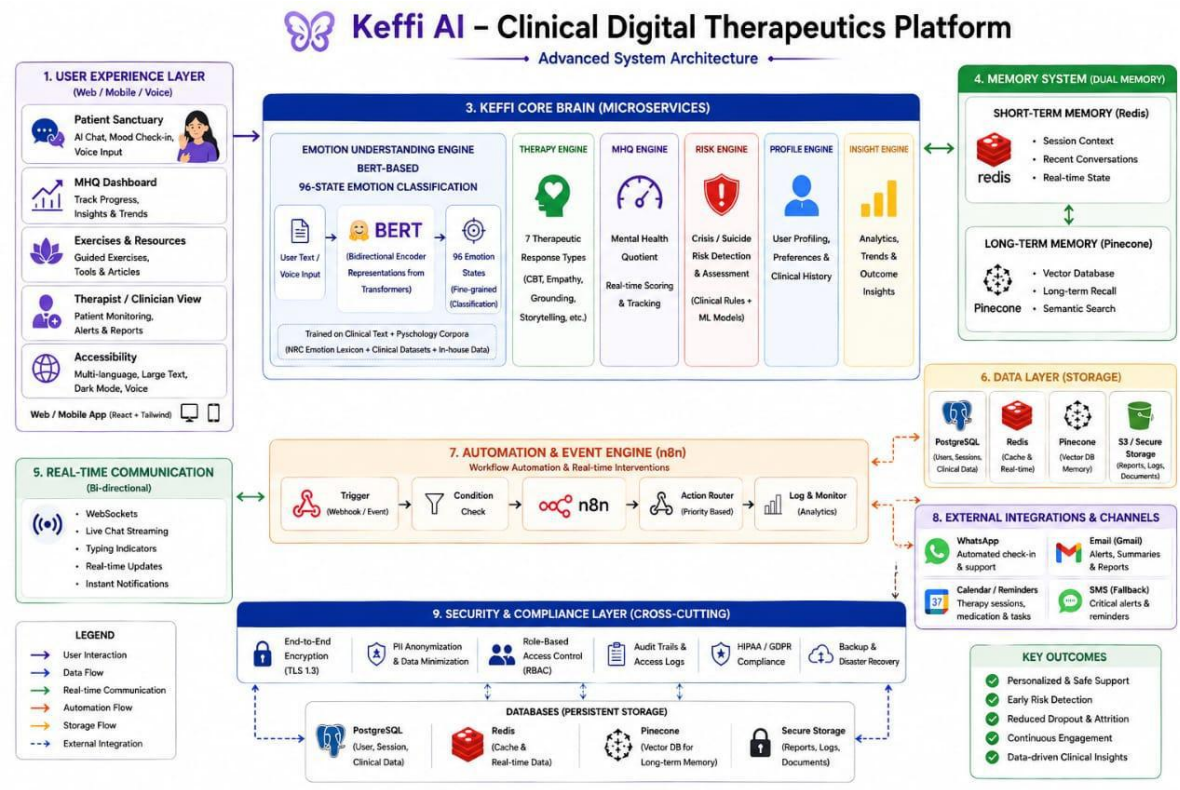
5.2 System Architecture

The architecture begins with user interaction through web or mobile interfaces. The user sends text or voice input to the system. The FastAPI backend receives the input and forwards it to the BERT-based NLP engine for emotion analysis.

5.2 System Architecture

The architecture begins with user interaction through web or mobile interfaces. The user sends text or voice input to the system. The FastAPI backend receives the input and forwards it to the BERT-based NLP engine for emotion analysis.

The BERT model identifies emotional states and sends the emotional data to the MHQ scoring module and Pinecone vector memory system. The memory system retrieves previous emotional conversations to maintain personalized interaction.



Based on emotional severity, the therapeutic response engine generates appropriate responses using CBT, grounding techniques, empathy-based communication, or crisis intervention strategies. Finally, the generated response is displayed to the user through the frontend interface.

The User Interaction Module acts as the communication layer

between the user and the system. It allows users to interact through text chat, voice input, mood check-ins, and emotional tracking features. The interface is developed using React.js and Tailwind CSS to provide a responsive and user-friendly experience.

This module collects user messages and forwards them to the backend for emotional analysis. It also displays therapeutic responses, emotional insights, MHQ scores, and chatbot conversations in real time.

Functions:

• User login and authentication

• Text and voice communication

• Mood tracking dashboard

• Real-time chatbot interaction

• Emotional progress visualization

5.3.2 Emotion Detection Module

The Emotion Detection Module is the core AI component of the system. It uses a BERT-based transformer model to analyze user conversations and classify emotional states. Unlike traditional sentiment analysis systems, this module identifies 96 clinically relevant emotional states related to depression, anxiety, stress, emotional burnout, and psychological distress.

The module processes textual input using NLP techniques and generates emotional confidence scores. These emotional predictions help the system understand the user's mental condition more accurately.

Functions:

• NLP preprocessing

• Emotion classification

• Multi-emotion detection

• Confidence score generation

• Emotional severity analysis

5.3.3 Therapeutic Response Module

The Therapeutic Response Module generates emotionally supportive and clinically safe responses based on the detected emotional condition. The module uses psychological frameworks such as Cognitive Behavioral Therapy (CBT), grounding exercises, reflective questioning, empathy-based communication, and Acceptance & Commitment Therapy (ACT).

The system dynamically selects the most suitable therapeutic strategy according to the user's emotional state and mental health condition.

Functions:

• CBT-based responses

• Empathy and validation replies

• Grounding techniques

• Reflective questioning

• Crisis intervention support

5.3.4 Memory Management Module

The Memory Management Module stores and retrieves previous emotional conversations using Pinecone vector databases and Redis memory systems. This module enables the chatbot to remember user history, emotional triggers, behavioral changes, and previous interactions.

Unlike ordinary chatbots, this module provides long-term emotional continuity and personalized interaction by retrieving emotionally similar conversations whenever needed.

Functions:

• Short-term conversation memory

• Long-term emotional memory

• Vector embedding storage

• Emotional history retrieval

• Personalized interaction support

5.3.5 Mental Health Analysis Module

The Mental Health Analysis Module continuously evaluates the user's emotional condition using the Mental Health Quotient (MHQ) scoring system. It calculates emotional severity levels and predicts psychological risk conditions such as depression, emotional decline, stress escalation, and suicidal ideation.

The module helps in continuous emotional monitoring and early mental health intervention.

Functions:

• MHQ score calculation

• Emotional risk prediction

• Depression severity analysis

• Attrition detection

• Crisis identification

5.3.6 Automation & Notification Module

The Automation Module uses n8n workflows to automate engagement and emergency communication processes. Whenever high-risk emotional conditions are detected, the module automatically triggers reminders, WhatsApp alerts, doctor notifications, and crisis response workflows.

This module improves patient engagement and reduces treatment dropout.

Functions:

• Automated reminders

• WhatsApp notifications

• Crisis alert activation

• Follow-up engagement

• Workflow automation

5.3.7 Database & Security Module

This module handles secure storage and protection of user information, emotional data, reports, and conversation history. PostgreSQL is used for structured data storage, while Redis and Pinecone handle caching and vector memory operations. Security mechanisms such as encryption, authentication, and anonymization ensure safe handling of sensitive mental health information.

Functions:

• User data storage

• Secure authentication

• Encrypted communication

• Database management

• Privacy protection and anonymize

5.3.8 High-Concurrency SQLite Write-Ahead Logging (WAL Mode Engine)

To eliminate database locking errors during concurrent multi-threaded requests, `clinical_db.py` configures SQLite with Write-Ahead Logging (`PRAGMA journal_mode=WAL`) and `PRAGMA synchronous=NORMAL`.

5.3.9 Patient Profile Registration & Schema Specs (`POST /api/register`)

Upserts patient profiles into the `patients` table storing `patient_id` (P-Phone), `name`, `phone`, `email`, `dob`, `gender`, `place`, `mhq_score`, `depression_level`, `assigned_doctor`, `created_at`, `last_active_at`.

5.3.10 User-Wise Isolated Chat History Persistence (`GET /api/history/{patient_id}`)

Stores messages isolated by `patient_id` in `chat_messages`, restoring the patient's past conversation timeline automatically upon login.

5.3.11 Admin Clinical Hub A-to-Z Patient Tracking & Transcript Inspection

Endpoints `GET /api/admin/patients_full` and `GET /api/admin/patient_detail/{patient_id}` compute Days Inactive metrics and present a complete scrollable conversation transcript viewer.

CHAPTER 6

IMPLEMENTATION

```
import React, { useState, useEffect, useRef } from 'react';
import axios from 'axios';

// ================================================

// CUSTOM ICONS (SVG inline)

// ================================================

const Icon = ({ size = 24, className = "", children }) => (
<svg xmlns="http://www.w3.org/2000/svg" width={size} height={size} viewBox="0 0 24 24"
fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"

className={className}>{children}</svg>

);

const ArrowRight = (p) => <Icon {...p}><path d="M5 12h14M12 5l7 7-7 7"/></Icon>;
const Shield = (p) => <Icon {...p}><path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8
10z"/></Icon>;
const Sparkles = (p) => <Icon {...p}><path d="m12 3-1.912 5.813a2 2 0 0 1-1.275 1.275L3
12l5.813 1.912a2 2
0 0 1 1.275 1.275L12 21l1.912-5.813a2 2 0 0 1 1.275-1.275L21 12l-5.813-1.912a2 2 0 0 1-
1.275-1.275L12

3Z"/></Icon>;

const User = (p) => <Icon {...p}><path d="M19 21v-2a4 4 0 0 0-4-4H9a4 4 0 0 0-4
4v2"/><circle cx="12"
cy="7" r="4"/></Icon>;

const Users = (p) => <Icon {...p}><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4
4v2"/><circle cx="9"
cy="7" r="4"/><path d="M22 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1 0
7.75"/></Icon>;

const MessageCircle = (p) => <Icon {...p}><path d="M7.9 20A9 9 0 1 0 4 16.1L2
22Z"/></Icon>;
const Clock = (p) => <Icon {...p}><circle cx="12" cy="12" r="10"/><polyline points="12 6
12 12 16
14"/></Icon>;
```

```
const Activity = (p) => <Icon {...p}><polyline points="22 12 18 12 15 21 9 3 6 12 2
12"/></Icon>;
const TrendingDown = (p) => <Icon {...p}><polyline points="22 17 13.5 8.5 8.5 13.5 2
7"/><polyline
points="16 17 22 17 22 11"/></Icon>;

const Brain = (p) => <Icon {...p}><path d="M9.5 2A2.5 2.5 0 0 0 7 4.5v15a2.5 2.5 0 0 0
4.9 1 2.5 2.5 0 0 0 4.2
0 2.5 2.5 0 0 0 4.9-1v-15A2.5 2.5 0 0 0 18.5 2H9.5z"/><path d="M12 2v20"/></Icon>;

const Database = (p) => <Icon {...p}><ellipse cx="12" cy="5" rx="9" ry="3"/><path d="M21
12c0 1.66-4 3-9
3s-9-1.34-9-3"/><path d="M3 5v14c0 1.66 4 3 9 3s9-1.34 9-3V5"/></Icon>;

const Target = (p) => <Icon {...p}><circle cx="12" cy="12" r="10"/><circle cx="12"
cy="12" r="6"/><circle
cx="12" cy="12" r="2"/></Icon>;

const Zap = (p) => <Icon {...p}><polygon points="13 2 3 14 12 14 11 22 21 10 12 10 13
2"/></Icon>;
const Heart = (p) => <Icon {...p}><path d="M19 14c1.49-1.46 3-3.21 3-5.5A5.5 5.5 0 0 0
16.5 3c-1.76 0-3 .5-
4.5 2-1.5-1.5-2.74-2-4.5-2A5.5 5.5 0 0 0 2 8.5c0 2.3 1.5 4.05 3 5.5l7 7Z"/></Icon>;
```

// Patient Dashboard Icons

```
const BookOpen = (p) => <Icon {...p}><path d="M2 3h6a4 4 0 0 1 4 4v14a3 3 0 0 0-3-
3H2z"/><path d="M22
3h-6a4 4 0 0 0-4 4v14a3 3 0 0 1 3-3h7z"/></Icon>;

const TrendingUp = (p) => <Icon {...p}><polyline points="22 7 13.5 15.5 8.5 10.5 2
17"/><polyline points="16
7 22 7 22 13"/></Icon>;

const Mic = (p) => <Icon {...p}><path d="M12 2a3 3 0 0 0-3 3v7a3 3 0 0 0 6 0V5a3 3 0 0 0-
3-3Z"/><path
d="M19 10v2a7 7 0 0 1-14 0v-2"/><line x1="12" x2="12" y1="19" y2="22"/></Icon>;

const Send = (p) => <Icon {...p}><line x1="22" x2="11" y1="2" y2="13"/><polygon
points="22 2 15 22 11 13
2 9 22 2"/></Icon>;

const Star = (p) => <Icon {...p}><polygon points="12 2 15.09 8.26 22 9.27 17 14.14 18.18
21.02 12 17.77 5.82
21.02 7 14.14 2 9.27 8.91 8.26 12 2"/></Icon>;

const MapPin = (p) => <Icon {...p}><path d="M20 10c0 6-8 12-8 12s-8-6-8-12a8 8 0 0 1 16
0Z"/><circle
cx="12" cy="10" r="3"/></Icon>;

const Gift = (p) => <Icon {...p}><polyline points="20 12 20 22 4 22 4 12"/><rect x="2"
y="7" width="20"
height="5"/><line x1="12" x2="12" y1="22" y2="7"/><path d="M12 7H7.5a2.5 2.5 0 0 1 0-
5C11 2 12 7 12
7z"/><path d="M12 7h4.5a2.5 2.5 0 0 0 0-5C13 2 12 7 12 7z"/></Icon>;
```

```
const Search = (p) => <Icon {...p}><circle cx="11" cy="11" r="8"/><line x1="21"
x2="16.65" y1="21"
```
y2="16.65"/></Icon>;

```
const Bell = (p) => <Icon {...p}><path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-
9"/><path d="M13.73
```
21a2 2 0 0 1-3.46 0"/></Icon>;

```
const AlertTriangle = (p) => <Icon {...p}><path d="m21.73 18-8-14a2 2 0 0 0-3.48 0l-8
14A2 2 0 0 0 4
```
21h16a2 2 0 0 0 1.73-3Z"/><line x1="12" y1="9" x2="12" y2="13"/><line x1="12" y1="17"
x2="12.01"

y2="17"/></Icon>;

```
const PhoneCall = (p) => <Icon {...p}><path d="M22 16.92v3a2 2 0 0 1-2.18 2 19.79 19.79 0
0 1-8.63-3.07
```
19.5 19.5 0 0 1-6-6 19.79 19.79 0 0 1-3.07-8.67A2 2 0 0 1 4.11 2h3a2 2 0 0 1 2 1.72 12.84 12.84
0 0 0 .7 2.81 2

2 0 0 1-.45 2.11L8.09 9.91a16 16 0 0 0 6 6l1.27-1.27a2 2 0 0 1 2.11-.45 12.84 12.84 0 0 0
2.81.7A2 2 0 0 1 22

16.92z"/></Icon>;

```
const Settings = (p) => <Icon {...p}><circle cx="12" cy="12" r="3"/><path d="M19.4
15a1.65 1.65 0 0 0 .33
```
1.82l.06.06a2 2 0 0 1 0 2.83 2 2 0 0 1-2.83 0l-.06-.06a1.65 1.65 0 0 0-1.82-.33 1.65 1.65 0 0 0-1
1.51V21a2 2 0

0 1-2 2 2 2 0 0 1-2-2v-.09A1.65 1.65 0 0 0 9 19.4a1.65 1.65 0 0 0-1.82.33l-.06.06a2 2 0 0 1-2.83
0 2 2 0 0 1 0-

2.83l.06-.06a1.65 1.65 0 0 0 .33-1.82 1.65 1.65 0 0 0-1.51-1H3a2 2 0 0 1-2-2 2 2 0 0 1 2-
2h.09A1.65 1.65 0 0 0

4.6 9a1.65 1.65 0 0 0-.33-1.82l-.06-.06a2 2 0 0 1 0-2.83 2 2 0 0 1 2.83 0l.06.06a1.65 1.65 0 0 0
1.82.33H9a1.65

1.65 0 0 0 1-1.51V3a2 2 0 0 1 2-2 2 2 0 0 1 2 2v.09a1.65 1.65 0 0 0 1 1.51 1.65 1.65 0 0 0 1.82-
.33l.06-.06a2 2 0

0 1 2.83 0 2 2 0 0 1 0 2.83l-.06.06a1.65 1.65 0 0 0-.33 1.82V9a1.65 1.65 0 0 0 1.51 1H21a2 2 0 0
1 2 2 2 2 0 0

1-2 2h-.09a1.65 1.65 0 0 0-1.51 1z"/></Icon>;

```
const PieChart = (p) => <Icon {...p}><path d="M21.21 15.89A10 10 0 1 1 8 2.83"/><path
d="M22 12A10 10 0
```
0 0 12 2v10z"/></Icon>;

```
const Smile = (p) => <Icon {...p}><circle cx="12" cy="12" r="10"/><path d="M8 14s1.5 2 4
2 4-2 4-2"/><line
```
x1="9" y1="9" x2="9.01" y2="9"/><line x1="15" y1="9" x2="15.01" y2="9"/></Icon>;

```
const Frown = (p) => <Icon {...p}><circle cx="12" cy="12" r="10"/><path d="M16 16s-1.5-2-4-2-4 2-4
```
2"/><line x1="9" y1="9" x2="9.01" y2="9"/><line x1="15" y1="9" x2="15.01" y2="9"/></Icon>;

```
const Meh = (p) => <Icon {...p}><circle cx="12" cy="12" r="10"/><line x1="8" y1="15" x2="16"
```
y2="15"/><line x1="9" y1="9" x2="9.01" y2="9"/><line x1="15" y1="9" x2="15.01" y2="9"/></Icon>;

```
const theme = {
```
bg: 'bg-[#F2F9F6]',

textMain: 'text-[#2C5555]',

textDark: 'text-slate-800',

outset: 'bg-white shadow-[0_20px_40px_rgb(58,112,112,0.05)] border border-slate-100/50 rounded-[2rem]',

outsetHover: 'hover:shadow-[0_20px_60px_rgb(58,112,112,0.1)] hover:-translate-y-1 transition-all duration-

300',

btnTeal: 'bg-gradient-to-r from-[#3A7070] to-[#2C5555] text-white shadow-

[0_10px_20px_rgba(58,112,112,0.3)] hover:shadow-[0_15px_30px_rgba(58,112,112,0.4)] hover:-translate-y-1

transition-all border border-[#8FA989]/30',

btnOutline: 'border border-[#3A7070]/20 text-[#3A7070] shadow-[0_10px_20px_rgba(58,112,112,0.05)]

hover:border-[#3A7070]/40 hover:shadow-[0_15px_30px_rgba(58,112,112,0.1)] hover:-translate-y-1 transition-

all bg-white'

};

```
const KeffiLogo = ({ size = "w-10 h-10", onClick }) => (
<div onClick={onClick} className={`${size} relative flex items-center justify-center cursor-pointer group`}>
<div className="absolute inset-0 bg-[#3A7070] rounded-xl blur-[6px] group-hover:blur-[8px] transition-all
```
opacity-30">

```
<div className={`absolute inset-0 bg-transparent rounded-xl flex items-center justify-center z-10`}>
<Star size={size === "w-10 h-10" ? 28 : 36} className="text-[#3A7070] fill-[#3A7070]" />
</div>
</div>
```

);

```
const DynamicLoginIllustration = ({ step, className }) => {
return (
<svg viewBox="0 0 400 400" className={className} fill="none"
xmlns="http://www.w3.org/2000/svg">
<style>
{`
@keyframes scan-line {
0%, 100% { transform: translateY(0); opacity: 0; }
10% { opacity: 1; }
90% { opacity: 1; }
50% { transform: translateY(60px); }
}
`}
</style>
{/* Universal Background for all steps */}
<circle cx="200" cy="200" r="180" fill="#E6F0F0" opacity="0.6" />
<circle cx="200" cy="200" r="130" fill="white" opacity="0.8" />
<ellipse cx="200" cy="340" rx="90" ry="12" fill="#3A7070" opacity="0.2"
className="animate-pulse" />
{step === 1 && (
<g className="animate-bounce" style={{ animationDuration: '3s' }}>
{/* Step 1: Secure Identity & Trust (Glowing Keyhole Shield) */}
<path d="M200 60 L280 90 V180 C280 250 200 310 200 330 C200 310 120 250 120 180 V90 L200
60
Z" fill="white" stroke="#3A7070" strokeWidth="8" strokeLinejoin="round" />
<path d="M200 80 L260 100 V180 C260 230 200 280 200 295 C200 280 140 230 140 180 V100
L200 80
Z" fill="#F2F9F6" />
{/* Keyhole */}
<circle cx="200" cy="180" r="18" fill="#F43F5E" className="animate-pulse" />
<path d="M192 190 L188 220 H212 L208 190 Z" fill="#F43F5E" className="animate-pulse" />
{/* Trust Network Nodes */}
<circle cx="200" cy="220" r="80" fill="none" stroke="#8FA989" strokeWidth="2"
strokeDasharray="8
8" className="animate-spin" style={{animationDuration: '10s'}} />
<circle cx="100" cy="120" r="6" fill="#D4A373" className="animate-ping" />
<circle cx="300" cy="250" r="5" fill="#8FA989" />
<circle cx="130" cy="280" r="4" fill="#3A7070" />
</g>
```

)}

{step === 2 && (

```
<g className="animate-bounce" style={{ animationDuration: '3s' }}>
```

{/* Step 2: OTP Verification Scanner */}

```
<rect x="130" y="100" width="140" height="220" rx="20" fill="#2C5555" stroke="#3A7070"
```

strokeWidth="6" />

```
<rect x="140" y="110" width="120" height="200" rx="12" fill="#F2F9F6" />
```

{/* Phone Screen Elements */}

```
<rect x="175" y="120" width="50" height="6" rx="3" fill="#8FA989" opacity="0.5" />
<circle cx="200" cy="180" r="22" fill="white" stroke="#3A7070" strokeWidth="4" />
<path d="M190 175 V165 C190 155 210 155 210 165 V175" stroke="#8FA989" strokeWidth="4"
```

strokeLinecap="round" />

```
<circle cx="200" cy="183" r="4" fill="#F43F5E" />
<rect x="160" y="225" width="80" height="8" rx="4" fill="#E6F0F0" />
<rect x="160" y="240" width="60" height="8" rx="4" fill="#E6F0F0" />
<rect x="160" y="260" width="80" height="18" rx="9" fill="#3A7070" />
```

{/* Scanning Laser Line */}

```
<line x1="130" y1="160" x2="270" y2="160" stroke="#F43F5E" strokeWidth="2" opacity="0"
```

style={{animation: 'scan-line 3s infinite ease-in-out'}} />

```
<polygon points="130,160 270,160 250,180 150,180" fill="#F43F5E" opacity="0.08"
style={{animation:
```

'scan-line 3s infinite ease-in-out'}} />

{/* Floating OTP Bubble */}

```
<g className="animate-pulse" style={{ animationDuration: '2s' }}>
<path d="M250 120 C250 100 270 90 290 90 C310 90 320 100 320 120 C320 140 310 150 290 150
```

L270 160 L275 145 C260 140 250 130 250 120 Z" fill="#D4A373" />

```
<circle cx="275" cy="120" r="3" fill="white" />
<circle cx="285" cy="120" r="3" fill="white" />
<circle cx="295" cy="120" r="3" fill="white" />
</g>
<ellipse cx="200" cy="210" rx="100" ry="20" fill="none" stroke="#8FA989" strokeWidth="3"
```

opacity="0.6" strokeDasharray="10 10" transform="rotate(-15 200 210)" />

```
<circle cx="100" cy="140" r="5" fill="#8FA989" className="animate-pulse" />
<circle cx="300" cy="230" r="4" fill="#D4A373" className="animate-ping" />
</g>
```

)}

{step === 3 && (

```
<g className="animate-bounce" style={{ animationDuration: '3s' }}>
```

{/* Step 3: Natural Sanctuary (Geometric Character under Leaf) */}

```
<path d="M200 50 C320 50 360 150 280 230 C200 310 80 310 40 230 C0 150 80 50 200 50 Z"
```

fill="#8FA989" opacity="0.2" />

```
<path d="M200 80 C270 80 290 150 240 200 C190 250 110 250 90 200 C70 150 130 80 200 80 Z"
fill="#3A7070" opacity="0.1" />

<circle cx="160" cy="150" r="28" fill="#3A7070" />
<path d="M160 185 C185 185 205 205 205 240 L205 290 L115 290 L115 240 C115 205 135 185
160
185 Z" fill="#2C5555" />

<path d="M110 275 C70 275 60 305 90 315 L230 315 C260 305 250 275 210 275 Z"
fill="#3A7070" />
<path d="M210 205 L260 190 L285 215 L235 230 Z" fill="#D4A373" className="animate-pulse"
/>
<path d="M210 200 L260 185 L285 210 L235 225 Z" fill="white" />
<path d="M250 160 C235 160 220 145 220 130 C220 115 235 100 250 115 C265 100 280 115 280
130
C280 145 265 160 250 160 Z" fill="#F43F5E" className="animate-bounce"
style={{animationDuration: '2s'}}

/>

<circle cx="310" cy="130" r="6" fill="#D4A373" className="animate-ping" />
<circle cx="90" cy="110" r="5" fill="#8FA989" className="animate-pulse" />
<circle cx="270" cy="250" r="4" fill="#D4A373" />
</g>
)}

</svg>
);

};

// ================================================

// 1. ULTIMATE LANDING PAGE (Level 3 Design)

// ================================================

const LandingPage = ({ setView }) => {
return (
<>
<div className="min-h-screen bg-[#F2F9F6] overflow-x-hidden text-[#1E293B] font-inter
selection:bg-
[#3A7070] selection:text-white relative">

<div className="floating-blob w-[600px] h-[600px] bg-[#8FA989] top-[-100px] right-[-
200px]"></div>
<div className="floating-blob w-[800px] h-[800px] bg-[#E6F0F0] top-[800px] left-[-
400px]"></div>
<div className="floating-blob w-[600px] h-[600px] bg-[#3A7070] opacity-10 top-[2200px]
right-[-
200px]" style={{animationDelay: '2s'}}></div>

{/*   HEADER */}
```

```
<nav className="w-full px-6 md:px-12 py-6 flex justify-between items-center max-w-
[1400px] mx-auto
```
bg-transparent relative z-50">

```
<div className="flex items-center gap-3 cursor-pointer" onClick={() =>
setView('landing')}>
<KeffiLogo size="w-12 h-12" />
<span className="text-3xl font-poppins font-bold text-[#2C5555] tracking-tight">Keffi
AI</span>
</div>
<div className="hidden lg:flex items-center gap-10 font-inter text-[16px] font-medium
text-slate-500">
<button onClick={() => setView('landing')} className="text-slate-900 font-semibold
transition-
```
colors">Home</button>

```
<button onClick={() => {document.getElementById('story').scrollIntoView({behavior:
'smooth'})}}
```
className="hover:text-[#3A7070] transition-colors">Our Story</button>

```
<button onClick={() => setView('login-admin')} className="hover:text-[#3A7070]
transition-
```
colors">Clinical Hub</button>

```
<button onClick={() => setView('login-patient')} className="hover:text-[#3A7070]
transition-
```
colors">Sanctuary</button>

```
</div>
<div className="flex items-center gap-6">
<button onClick={() => setView('login-patient')} className="hidden md:block font-inter
text-[16px]
```
font-medium text-slate-600 hover:text-[#3A7070] transition-colors">

Log in

```
</button>
<button onClick={() => setView('login-patient')} className="px-8 py-3.5 rounded-full bg-
[#3A7070]
```
text-white font-inter font-semibold text-[16px] shadow-lg shadow-[#3A7070]/30 hover:bg-[#2C5555]

hover:scale-105 transition-all">

Get Started

```
</button>
</div>
</nav>
<main className="relative z-10 w-full">
```
{/* SECTION 1: THE HERO (Gradient Layout) */}

```
<section className="w-full min-h-[90vh] flex items-center relative overflow-hidden pt-24
pb-12">
```
{/* Soft Blue/Green Gradient Background */}

```
<div className="absolute inset-0 bg-gradient-to-br from-[#E6F0F0] via-white to-[#E8F4F8] z-
```
0">

```
<div className="absolute top-0 left-0 w-[600px] h-[600px] bg-[#3A7070]/5 rounded-full blur-[100px] z-
```
0">

```
<div className="absolute bottom-0 right-0 w-[600px] h-[600px] bg-blue-500/5 rounded-full blur-
```
[100px] z-0">

```
<div className="max-w-[1200px] mx-auto px-6 lg:px-12 flex flex-col lg:flex-row items-center gap-12
```
lg:gap-20 relative z-10 w-full">

```
<div className="flex-1 flex flex-col items-start text-left">
<div className="inline-flex items-center gap-2 px-4 py-2 rounded-full bg-white border border-
```
[#3A7070]/20 text-[#3A7070] font-semibold text-sm mb-4 shadow-sm">

```
<Sparkles size={16} className="text-[#3A7070]" /> Keffi is an Emotion Engine
</div>
<h1 className="h1-title font-poppins text-slate-900 mb-6 leading-[1.1]">
```
Bridging the <span className="text-transparent bg-clip-text bg-gradient-to-r from-[#3A7070] to-

blue-600 font-black">Invisible Gap
in Mental Healthcare.

```
</h1>
<p className="p-text max-w-lg mb-6 text-slate-600 text-lg leading-relaxed">
```
Keffi is your Emotionally Intelligent AI Companion. While traditional therapy supports you for one

hour a week, Keffi bridges the 167-hour gap. It deeply understands 96 emotional states, safely remembers your

journey, and delivers personalized, human-like therapeutic support exactly when you need it. A safe sanctuary

where advanced AI meets profound empathy.

```
</p>
<div className="flex flex-col sm:flex-row items-center gap-4 w-full sm:w-auto mt-2">
<button onClick={() => setView('login-patient')} className="group relative w-full sm:w-auto px-10
```
py-5 rounded-2xl bg-gradient-to-r from-[#3A7070] to-[#2C5555] text-white font-inter font-bold text-lg shadow-

[0_20px_40px_rgba(58,112,112,0.4)] hover:shadow-[0_20px_60px_rgba(58,112,112,0.6)] hover:-translate-y-1

transition-all overflow-hidden flex items-center justify-center gap-3">

```
<div className="absolute inset-0 bg-white/20 blur-md transform -skew-x-12 -translate-x-
full group-
```
hover:translate-x-full transition-transform duration-700 ease-out">

Enter Keffi Chat <ArrowRight size={20} className="transform group-hover:translate-x-1

transition-transform"/>

```
</button>
<button onClick={() => setView('login-admin')} className="group relative w-full sm:w-auto
px-10
```
py-5 rounded-2xl bg-white border border-[#3A7070]/10 text-[#3A7070] font-inter font-bold text-lg shadow-

[0_20px_40px_rgba(58,112,112,0.1)] hover:shadow-[0_20px_60px_rgba(58,112,112,0.2)] hover:border-

[#3A7070]/30 hover:-translate-y-1 transition-all flex items-center justify-center gap-3">

Doctor / Clinical Hub

```
</button>
</div>
</div>
<div className="flex-1 flex justify-center lg:justify-end relative w-full max-w-[600px]
mt-10 lg:mt-0">
<div className="relative w-80 h-80 flex items-center justify-center">
<div className="absolute inset-0 bg-[#3A7070] rounded-full blur-[80px] opacity-30
animate-
```
pulse">

```
<div className="absolute inset-4 bg-emerald-400 rounded-full blur-[40px] opacity-
20"></div>
<div className="relative z-10 w-56 h-56 rounded-full bg-gradient-to-br from-[#E6F0F0] to-
white
```
shadow-[inset_-10px_-

10px_20px_rgba(0,0,0,0.05),_inset_10px_10px_20px_rgba(255,255,255,0.8),_0_20px_40px_rgba(58,112,112,0

.4)] flex items-center justify-center border-[8px] border-white group">

```
<div className="absolute inset-4 bg-[#3A7070] rounded-full blur-xl opacity-20 group-
```
hover:opacity-40 transition-opacity duration-700">

```
<Star size={96} className="text-[#3A7070] fill-[#3A7070] relative z-10 animate-
```
[spin_10s_linear_infinite] drop-shadow-[0_15px_15px_rgba(58,112,112,0.6)]" />

```
</div>
```
{/* Orbiting elements */}

```
<div className="absolute top-0 right-10 w-16 h-16 bg-white rounded-full shadow-xl flex
items-
```
center justify-center border border-emerald-100 text-emerald-500 transform hover:scale-110 transition-

transform z-20"><Heart size={28} className="fill-emerald-100"/>

```
<div className="absolute bottom-10 left-0 w-20 h-20 bg-white rounded-full shadow-xl flex
items-
```

center justify-center border border-slate-100 text-[#3A7070] transform hover:scale-110 transition-transform z-

20"><MessageCircle size={36} className="fill-slate-50"/>

```
</div>
</div>
</div>
</section>
```

{/* SECTION 2: THE PROBLEM (Story Zig-Zag) */}

```
<section id="story" className="scroll-3d w-full py-32 max-w-[1200px] mx-auto px-6 lg:px-
12 flex flex-
```

col lg:flex-row-reverse items-center gap-24 relative transition-all duration-75 ease-out">

```
<div className="absolute top-10 left-0 text-[300px] font-poppins font-bold text-slate-100
opacity-50 z-
```

[-1] leading-none select-none">01

```
<div className="flex-1 flex justify-center lg:justify-start">
<img src="https://illustrations.popsy.co/amber/surreal-hourglass.svg" alt="Hourglass
Drawing"
```

className="w-full max-w-[500px] drop-shadow-xl" />

```
</div>
<div className="flex-1 flex flex-col items-start text-left">
<h2 className="h2-title font-poppins text-slate-900 mb-8">The Silent Crisis We
Ignore.</h2>
<div className="space-y-6">
<p className="p-text">
```

Therapy typically happens for one hour a week. But emotional struggles don't follow a schedule. What

happens during the remaining 167 hours? Patients are left alone to fight their anxiety, burnout, and depression in

silence.

```
</p>
<p className="p-text">
```

Healing is incredibly hard, and it isn't linear. Due to stigma, high costs, and a lack of immediate

support, nearly 60% of patients drop out of treatment before fully recovering.

```
</p>
<p className="p-text">
```

When they drop out, relapses go completely undetected. Doctors have no proactive way to monitor

these at-risk patients outside the clinic walls. This is the gap Keffi AI was built to close.

```
</p>
</div>
</div>
</section>
```
{/* NEW SECTION 3: HOW KEFFI WORKS (Timeline Layout) */}

```
<section className="scroll-3d w-full py-32 bg-white border-y border-slate-100 transition-
all duration-75
```
ease-out">

```
<div className="max-w-[1200px] mx-auto px-6 lg:px-12 text-center">
<h2 className="h2-title font-poppins text-slate-900 mb-20">How Keffi Heals.</h2>
<div className="relative flex flex-col items-center">
```
{/* Vertical Line */}

```
<div className="absolute top-0 bottom-0 left-1/2 -translate-x-1/2 w-1 bg-gradient-to-b
from-
```
[#3A7070]/20 via-[#8FA989]/20 to-transparent">

{/* Step 1 */}

```
<div className="w-full flex justify-between items-center mb-24 relative">
<div className="w-[45%] text-right pr-10">
<h3 className="h3-title font-poppins text-slate-900 mb-3">1. You Express.</h3>
<p className="p-small">Enter the Sanctuary whenever anxiety hits. Type or speak your
thoughts
```
into Keffi exactly as you feel them, without fear of judgment.</p>

```
</div>
<div className="w-16 h-16 rounded-full bg-white border-4 border-[#3A7070] flex items-
center
```
justify-center text-[#3A7070] z-10 shadow-lg font-bold text-xl">1

```
<div className="w-[45%] text-left pl-10 opacity-50"><MessageCircle size={64}/></div>
</div>
```
{/* Step 2 */}

```
<div className="w-full flex justify-between items-center mb-24 relative">
<div className="w-[45%] text-right pr-10 opacity-50 flex justify-end"><Brain
size={64}/></div>
<div className="w-16 h-16 rounded-full bg-[#3A7070] border-4 border-white flex items-
center
```
justify-center text-white z-10 shadow-lg font-bold text-xl">2

```
<div className="w-[45%] text-left pl-10">
<h3 className="h3-title font-poppins text-slate-900 mb-3">2. Keffi Analyzes.</h3>
<p className="p-small">Behind the scenes, Keffi's BERT engine detects your exact
emotional
```
state across 96 fine-grained categories, pulling from your Pinecone memory history.</p>

```
</div>
</div>
```
{/* Step 3 */}

```
<div className="w-full flex justify-between items-center relative">
<div className="w-[45%] text-right pr-10">
```

```
<h3 className="h3-title font-poppins text-slate-900 mb-3">3. Immediate Relief.</h3>
<p className="p-small">Keffi dynamically responds using 1 of 7 therapeutic modes—from
```
guiding a breathing exercise to reframing negative thoughts using CBT principles.</p>

```
</div>
<div className="w-16 h-16 rounded-full bg-[#8FA989] border-4 border-white flex items-
center
```
justify-center text-white z-10 shadow-lg font-bold text-xl">3

```
<div className="w-[45%] text-left pl-10 opacity-50"><Heart size={64}/></div>
</div>
</div>
</div>
</section>
```
{/* NEW SECTION 4: THE SCIENCE (Grid Layout) */}

```
<section className="scroll-3d w-full py-32 max-w-[1200px] mx-auto px-6 lg:px-12 flex
flex-col lg:flex-
```
row items-center gap-24 relative transition-all duration-75 ease-out">

```
<div className="absolute top-10 right-10 text-[300px] font-poppins font-bold text-slate-
100 opacity-50
```
z-[-1] leading-none select-none">02

```
<div className="flex-1 grid grid-cols-2 gap-6 relative">
<div className="absolute -inset-4 bg-gradient-to-tr from-[#3A7070]/10 to-[#8FA989]/10
rounded-
```
[3rem] blur-xl z-[-1]">

```
<div className="bg-white p-8 rounded-3xl shadow-lg flex flex-col gap-4 transform
translate-y-8">
<div className="w-12 h-12 bg-indigo-50 text-indigo-500 rounded-full flex items-center
justify-
```
center"><Activity size={24}/>

```
<h4 className="font-poppins font-bold text-lg text-slate-800">96-State BERT</h4>
<p className="text-sm text-slate-500">Not just "sad". Keffi detects complex states like
Atypical
```
Depression.</p>

```
</div>
<div className="bg-white p-8 rounded-3xl shadow-lg flex flex-col gap-4">
<div className="w-12 h-12 bg-amber-50 text-amber-500 rounded-full flex items-center
justify-
```
center"><Database size={24}/>

```
<h4 className="font-poppins font-bold text-lg text-slate-800">Pinecone Memory</h4>
<p className="text-sm text-slate-500">Vector memory ensures Keffi never forgets your past
triggers
```
or progress.</p>

```
</div>
<div className="bg-white p-8 rounded-3xl shadow-lg flex flex-col gap-4 transform
translate-y-8">
<div className="w-12 h-12 bg-emerald-50 text-emerald-500 rounded-full flex items-center
justify-
```

center"><Target size={24}/>

```
<h4 className="font-poppins font-bold text-lg text-slate-800">Dynamic MHQ</h4>
<p className="text-sm text-slate-500">Silent, continuous evaluation of your Mental Health
Quotient
```

during chats.</p>

```
</div>
<div className="bg-white p-8 rounded-3xl shadow-lg flex flex-col gap-4">
<div className="w-12 h-12 bg-rose-50 text-rose-500 rounded-full flex items-center
justify-
```

center"><Zap size={24}/>

```
<h4 className="font-poppins font-bold text-lg text-slate-800">7-Mode Engine</h4>
<p className="text-sm text-slate-500">Switches seamlessly between active listening, CBT
reframing,
```

and crisis mode.</p>

```
</div>
</div>
<div className="flex-1 flex flex-col items-start text-left">
<h2 className="h2-title font-poppins text-slate-900 mb-8">An Engine Built on
Empathy.</h2>
<div className="space-y-6">
<p className="p-text">
```

Behind the calming interface lies a robust Triple-AI Architecture. We don't rely on simple prompts.

Keffi is powered by a custom-trained clinical classification system that understands the deepest nuances of

human emotion.

```
</p>
<p className="p-text">
```

By combining semantic search with real-time generative capabilities, Keffi replaces tedious weekly

assessment forms with natural, empathetic dialogue. It learns your unique psychological profile to offer deeply

personalized care.

```
</p>
</div>
</div>
</section>
```

{/* SECTION 5: DUAL PLATFORM (True Split-Screen) */}

```
<section className="scroll-3d w-full flex flex-col lg:flex-row mt-20 border-y border-
slate-200 transition-
```

all duration-75 ease-out">

{/* Left Side: Patient (Light) */}

```
<div className="flex-1 bg-[#F2F9F6] p-16 lg:p-24 flex flex-col items-center text-center">
<div className="h-48 flex items-end justify-center mb-10">
<img src="https://illustrations.popsy.co/amber/success.svg" alt="Patient Sanctuary"
className="h-full
```
drop-shadow-xl" />
```
</div>
<h2 className="h2-title font-poppins text-slate-900 mb-6">For Patients.<br/>The
Sanctuary.</h2>
<p className="p-text text-slate-600 mb-10 max-w-sm">
```
A completely judgment-free zone to vent, track your shifting moods, and receive real-time emotional

first-aid without having to wait weeks for an appointment.
```
</p>
<button onClick={() => setView('login-patient')} className={`mt-auto px-10 py-4 rounded-2xl font-
```
inter font-bold text-lg ${theme.btnTeal}`}>

Keffi
```
</button>
</div>
```
{/* Right Side: Doctor (Light) */}
```
<div className="flex-1 bg-white p-16 lg:p-24 flex flex-col items-center text-center
border-l border-
```
slate-100">
```
<div className="h-48 flex items-end justify-center mb-10">
<img src="https://illustrations.popsy.co/amber/video-call.svg" alt="Clinical Hub"
className="h-full
```
drop-shadow-2xl" />
```
</div>
<h2 className="h2-title font-poppins text-slate-900 mb-6">For Clinicians.<br/>The
Hub.</h2>
<p className="p-text text-slate-600 mb-10 max-w-sm">
```
A predictive dashboard giving you a real-time view of your entire roster's emotional trajectory. If a

patient shows signs of relapse, Keffi alerts you instantly.
```
</p>
<button onClick={() => setView('login-admin')} className={`mt-auto px-10 py-4 rounded-2xl font-
```
inter font-bold text-lg ${theme.btnOutline}`}>

Access Hub
```
</button>
</div>
</section>
```
{/* SECTION 6: SAFETY (Light Mode Alert Block) */}

```
<section className="scroll-3d w-full bg-[#F2F9F6] py-32 relative overflow-hidden border-y
border-red-
```
500/10 transition-all duration-75 ease-out">

```
<div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[800px] h-
[800px] bg-red-
```
500/5 rounded-full blur-[120px]">

```
<div className="max-w-[800px] mx-auto px-6 text-center relative z-10 flex flex-col items-
center">
<div className="w-24 h-24 mb-10 text-red-500 animate-pulse"><Shield size={96}
strokeWidth={1}
```
/>

```
<h2 className="h2-title font-poppins text-slate-900 mb-8">Clinical Safety First.
Always.</h2>
<p className="p-text text-slate-600 max-w-3xl">
```
Built with strict WHO-compliant crisis protocols and the n8n automation engine, Keffi continuously

monitors conversations for high-risk signals. In moments of severe distress or suicidal ideation, it overrides AI

independence and instantly triggers an SOS alert to emergency contacts and your clinical supervisor.

```
<br/><br/>
<span className="text-red-500 font-semibold">We prioritize human safety over everything
```
else.

```
</p>
</div>
</section>
```
{/* FINAL CTA */}

```
<section className="w-full py-32 bg-[#F2F9F6] flex flex-col items-center text-center
relative overflow-
```
hidden">

```
<div className="w-full max-w-[1000px] mx-auto bg-gradient-to-br from-[#E6F0F0] to-
[#FDFCF8]
```
border border-slate-200 rounded-[3rem] p-16 lg:p-24 shadow-xl relative overflow-hidden">

```
<div className="absolute top-0 right-0 w-[400px] h-[400px] bg-[#3A7070]/5 rounded-full
blur-
```
[80px]">

```
<h2 className="h1-title font-poppins text-slate-900 mb-6 relative z-10">Ready to
Heal?</h2>
<p className="p-text text-slate-600 mb-12 relative z-10 max-w-lg mx-auto">Whether you are
seeking
```
support or providing care, Keffi AI is here to bridge the gap.</p>

```
<button onClick={() => setView('login-patient')} className={`px-12 py-5 rounded-2xl font-
inter font-
```

bold text-xl relative z-10 ${theme.btnTeal}`}>

Keffi

```
</button>
</div>
</section>
</main>
```

{/* ULTIMATE FOOTER */}

```
<footer className="w-full relative z-10 bg-[#E6F0F0] text-slate-900 overflow-hidden mt-10
border-t
```

border-[#3A7070]/10">

{/* Glow Effects */}

```
<div className="absolute top-0 left-1/2 -translate-x-1/2 w-full max-w-[1000px] h-px bg-
gradient-to-r
```

from-transparent via-[#3A7070]/50 to-transparent">

```
<div className="absolute top-0 right-0 w-[500px] h-[500px] bg-white rounded-full blur-
[100px] pointer-
```

events-none">

```
<div className="absolute bottom-0 left-0 w-[500px] h-[500px] bg-emerald-100/50 rounded-
full blur-
```

[100px] pointer-events-none">

```
<div className="max-w-[1200px] mx-auto px-6 lg:px-12 pt-24 pb-12 relative z-10">
<div className="grid grid-cols-1 md:grid-cols-12 gap-16 lg:gap-8 mb-20">
```

{/* Column 1: Brand & Emergency */}

```
<div className="col-span-1 md:col-span-5 flex flex-col items-start">
<div className="flex items-center gap-3 mb-8 bg-white/60 px-6 py-4 rounded-2xl border
border-
```

white backdrop-blur-sm w-fit shadow-sm">

```
<Star size={32} className="text-[#3A7070] fill-[#3A7070]" />
<span className="text-3xl font-poppins font-bold tracking-tight text-[#2C5555]">Keffi
AI</span>
</div>
<p className="font-inter text-slate-600 text-lg mb-8 max-w-sm leading-relaxed">
```

Bridging the invisible gap in mental healthcare with continuous, empathetic AI tracking.

```
</p>
<div className="p-6 rounded-2xl bg-white/80 border border-red-200 backdrop-blur-sm w-full
max-w-
```

md shadow-sm">

```
<div className="flex items-center gap-2 mb-3">
<Shield size={20} className="text-red-500" />
<h4 className="font-poppins font-semibold text-red-600 uppercase tracking-widest text-
```

xs">Emergency Support</h4>

```
</div>
<p className="font-inter text-slate-700 text-sm leading-relaxed">
```

If you are in a life-threatening situation, please call local emergency services immediately. This platform is not a substitute for emergency medical care.

```
</p>
</div>
</div>
```

{/* Column 2: Navigation */}

```
<div className="col-span-1 md:col-span-3 lg:col-span-2">
<h4 className="font-poppins font-semibold text-slate-900 mb-6 uppercase tracking-wider text-
```

sm">Platform</h4>

```
<ul className="flex flex-col gap-4 font-inter text-slate-600">
<li><button onClick={() => setView('login-patient')} className="hover:text-[#3A7070] font-
```

medium transition-colors">Patient Sanctuary</button>

```
<li><button onClick={() => setView('login-admin')} className="hover:text-[#3A7070] font-medium
```

transition-colors">Clinical Hub</button>

```
<li><button onClick={() => {document.getElementById('story')?.scrollIntoView({behavior:
```

'smooth'})}} className="hover:text-[#3A7070] font-medium transition-colors">How it Works</button>

```
</ul>
</div>
```

{/* Column 3: Legal */}

```
<div className="col-span-1 md:col-span-4 lg:col-span-5 flex flex-col lg:items-end lg:text-right">
<h4 className="font-poppins font-semibold text-slate-900 mb-6 uppercase tracking-wider text-
```

sm">Project Details</h4>

```
<p className="font-inter text-slate-700 font-medium text-xl mb-2">
```

Naan Mudhalvan - Niral Thiruvizha

```
</p>
<p className="font-inter text-[#3A7070] font-bold text-lg mb-10">
```

Built with by Team Keffi @ UCE Panruti

```
</p>
<div className="flex flex-wrap gap-6 font-inter text-sm text-slate-500 mt-auto justify-start lg:justify-
```

end">

```
<a href="#" className="hover:text-slate-900 transition-colors">Privacy Policy</a>
<a href="#" className="hover:text-slate-900 transition-colors">Terms of Service</a>
<a href="#" className="hover:text-slate-900 transition-colors">Clinical Guidelines</a>
</div>
</div>
</div>
```

```
<div className="pt-8 border-t border-[#3A7070]/10 flex flex-col md:flex-row justify-
between items-
```
center gap-4 text-slate-500 text-sm font-inter">

```
<p>© 2026 Keffi AI Platform. All rights reserved.</p>
<p className="flex items-center gap-2">Version 3.0.0 <span className="w-2 h-2 rounded-
full bg-
```
emerald-500 animate-pulse"> Systems Operational</p>

```
</div>
</div>
</footer>
</div>
</>
```
);

};

// ==

// 2. PATIENT ONBOARDING

// ==

```
const PatientLogin = ({ setView, setUserData }) => {
const [step, setStep] = useState(1);
const [formData, setFormData] = useState({ phone: '', email: '', otp: '', name: '', age:
'', dob: '', gender: '', place: ''
```
});

```
const [error, setError] = useState('');
const handleNext = () => {
```
setError('');

if (step === 1) {

```
const phoneClean = formData.phone.replace(/\D/g, '');
```
if (!formData.phone) return setError('Please enter your mobile number.');

if (phoneClean.length < 10) return setError('Mobile number must be at least 10 digits.');

if (!formData.email) return setError('Please enter your email address.');

if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email)) return setError('Please enter a valid email

address.');

setStep(2);

} else if (step === 2) {

if (!formData.otp) return setError('Please enter the 4-digit code.');

if (formData.otp.length !== 4) return setError('Code must be exactly 4 digits.');

```
setStep(3);
} else if (step === 3) {
if (!formData.name.trim()) return setError('Please enter your preferred name.');
if (!formData.age || isNaN(formData.age) || +formData.age < 5 || +formData.age > 120) return
setError('Please enter a valid age (5–120).');
if (!formData.gender) return setError('Please select your gender.');
const phoneClean = formData.phone.replace(/\D/g, '');
const patientId = 'P-' + phoneClean.slice(-6);
const fullData = { ...formData, patient_id: patientId };
localStorage.setItem('keffi_user', JSON.stringify(fullData));
setUserData(fullData);
setView('patient-dashboard');
}
};
return (
<div className={`min-h-screen flex items-center justify-center p-6 md:p-10`}>
<div className={`max-w-5xl w-full h-[700px] rounded-[2.5rem] bg-white shadow-
[0_20px_60px_rgba(0,0,0,0.08)] flex flex-col md:flex-row overflow-hidden border border-slate-
100`}>
{/* Left Image Side */}
<div className="hidden md:flex md:w-5/12 relative overflow-hidden bg-[#E6F0F0] rounded-l-
[2.5rem]
items-center justify-center p-8">
<DynamicLoginIllustration step={step} className="w-full h-full max-w-[300px] object-
contain relative
z-10 drop-shadow-2xl transition-all duration-500" />
<div className="absolute inset-0 bg-gradient-to-t from-[#2C5555]/40 via-transparent to-
transparent flex
flex-col justify-end p-12 text-[#1E293B] z-20">
<h2 className="text-3xl font-black mb-3">
{step === 1 && "Secure Entry"}
{step === 2 && "Verify Identity"}
{step === 3 && "Your Sanctuary"}
</h2>
<p className="text-base opacity-90 font-medium">
{step === 1 && "Enter your details below to establish a secure connection."}
{step === 2 && "We have sent an encrypted OTP. Please enter it below."}
```

{step === 3 && "Let's build your personal profile for a better experience."}

```
</p>
</div>
</div>
```

{/* Right Form Side */}

```
<div className="w-full md:w-7/12 p-10 md:p-16 flex flex-col justify-center bg-white">
<div className="flex justify-start gap-3 mb-10">
```

{[1,2,3].map(i => (

```
<div key={i} className={`h-1.5 rounded-full transition-all duration-500 ${i === step ?
'w-12 bg-
```

[#3A7070]' : 'w-4 bg-slate-200'}`}>

))}

```
</div>
```

{step === 1 && (

```
<div className="space-y-8 animate-fade-in">
<div>
<h2 className="text-3xl font-black text-slate-800 mb-3">Secure Entry</h2>
<p className="text-slate-500 text-base">Your privacy is our priority. Enter details for a
secure
```

OTP.</p>

```
</div>
<div className="space-y-5">
<div>
<label className="block text-sm font-bold text-slate-700 mb-2">Mobile Number</label>
<input type="tel" placeholder="+91 98765 43210" value={formData.phone} onChange={e =>
```

setFormData({...formData, phone: e.target.value})} className={`w-full p-4 rounded-xl bg-slate-50 border

border-slate-200 outline-none text-slate-800 font-medium text-base focus:border-[#3A7070] focus:ring-4

focus:ring-[#3A7070]/10 transition-all`} />

```
</div>
<div>
<label className="block text-sm font-bold text-slate-700 mb-2">Email Address</label>
<input type="email" placeholder="you@example.com" value={formData.email} onChange={e =>
```

setFormData({...formData, email: e.target.value})} className={`w-full p-4 rounded-xl bg-slate-50 border

border-slate-200 outline-none text-slate-800 font-medium text-base focus:border-[#3A7070] focus:ring-4

focus:ring-[#3A7070]/10 transition-all`} />

```
</div>
</div>
```

{error && <div className="text-red-500 text-sm font-bold bg-red-50 border border-red-200 rounded-

xl px-4 py-3">{error} }

```
<div className="pt-2 space-y-4">
<button onClick={handleNext} className={`w-full py-4 rounded-xl font-bold text-base
```
${theme.btnTeal}`}>Get Magic Code</button>

```
<button onClick={() => setView('landing')} className="w-full text-center text-slate-400
font-bold
```
text-sm hover:text-[#3A7070] transition-colors">Back to Home</button>

```
</div>
</div>
```
)}

{step === 2 && (

```
<div className="space-y-8 animate-fade-in">
<div>
<h2 className="text-3xl font-black text-slate-800 mb-3">Verification</h2>
<p className="text-slate-500 text-base">Enter the 4-digit code sent to <span
className="text-slate-
```
800 font-bold">{formData.phone}</p>

```
</div>
<input type="number" placeholder="0000" value={formData.otp} onChange={e =>
```
setFormData({...formData, otp: e.target.value})} className={`w-full p-6 text-center text-5xl tracking-[0.5em]

rounded-2xl bg-slate-50 border border-slate-200 outline-none text-[#3A7070] font-black focus:border-

[#3A7070] focus:ring-4 focus:ring-[#3A7070]/10 transition-all`} maxLength={4} />

{error && <div className="text-red-500 text-sm font-bold bg-red-50 border border-red-200 rounded-

xl px-4 py-3">{error} }

```
<div className="pt-4">
<button onClick={handleNext} className={`w-full py-4 rounded-xl font-bold text-base
```
${theme.btnTeal}`}>Verify Identity</button>

```
</div>
</div>
```
)}

{step === 3 && (

```
<div className="space-y-8 animate-fade-in h-full flex flex-col justify-center">
<div>
<h2 className="text-3xl font-black text-slate-800 mb-3">Your Profile</h2>
<p className="text-slate-500 text-base">Help Keffi understand you better.</p>
```

```
</div>
<div className="space-y-5">
<div>
<label className="block text-sm font-bold text-slate-700 mb-2">Preferred Name</label>
<input type="text" placeholder="What should we call you?" value={formData.name}
onChange={e
```
=> setFormData({...formData, name: e.target.value})} className={`w-full p-4 rounded-xl bg-slate-50 border

border-slate-200 outline-none text-slate-800 font-medium text-base focus:border-[#3A7070] focus:ring-4

focus:ring-[#3A7070]/10 transition-all`} />

```
</div>
<div className="flex gap-5">
<div className="w-1/3">
<label className="block text-sm font-bold text-slate-700 mb-2">Age</label>
<input type="number" placeholder="Age" value={formData.age} onChange={e =>
```
setFormData({...formData, age: e.target.value})} className={`w-full p-4 rounded-xl bg-slate-50 border border-

slate-200 outline-none text-slate-800 font-medium text-base focus:border-[#3A7070] focus:ring-4 focus:ring-

[#3A7070]/10 transition-all`} />

```
</div>
<div className="w-2/3">
<label className="block text-sm font-bold text-slate-700 mb-2">Date of Birth</label>
<input type="date" value={formData.dob} onChange={e => setFormData({...formData, dob:
```
e.target.value})} className={`w-full p-4 rounded-xl bg-slate-50 border border-slate-200 outline-none text-

slate-600 font-medium text-base focus:border-[#3A7070] focus:ring-4 focus:ring-[#3A7070]/10 transition-all`}

/>

```
</div>
</div>
<div className="flex gap-5">
<div className="w-1/2">
<label className="block text-sm font-bold text-slate-700 mb-2">Gender</label>
<select value={formData.gender} onChange={e => setFormData({...formData, gender:
```
e.target.value})} className={`w-full p-4 rounded-xl bg-slate-50 border border-slate-200 outline-none text-

slate-800 font-medium text-base focus:border-[#3A7070] focus:ring-4 focus:ring-[#3A7070]/10 transition-all

appearance-none`}>

```
<option value="" disabled>Select</option>
<option value="Male">Male</option>
<option value="Female">Female</option>
<option value="Non-binary">Non-binary</option>
<option value="Prefer not to say">Prefer not to say</option>
</select>
</div>
<div className="w-1/2">
<label className="block text-sm font-bold text-slate-700 mb-2">Location</label>
<input type="text" placeholder="City / District" value={formData.place} onChange={e =>
```
setFormData({...formData, place: e.target.value})} className={`w-full p-4 rounded-xl bg-slate-50 border

border-slate-200 outline-none text-slate-800 font-medium text-base focus:border-[#3A7070] focus:ring-4

focus:ring-[#3A7070]/10 transition-all`} />

```
</div>
</div>
</div>
```
{error && <div className="text-red-500 text-sm font-bold bg-red-50 border border-red-200 rounded-

xl px-4 py-3">{error} }

```
<div className="pt-2 mt-auto">
<button onClick={handleNext} className={`w-full py-4 rounded-xl font-bold text-base
```
${theme.btnTeal}`}>Enter Keffi</button>

```
</div>
</div>
```
)}

```
</div>
</div>
</div>
```
);

};

// ==

// 3. ADMIN LOGIN

// ==

```
const AdminLogin = ({ setView }) => {
return (
<div className={`min-h-screen flex items-center justify-center p-6`}>
<div className={`max-w-md w-full rounded-[2rem] ${theme.outset} p-10 flex flex-col gap-
8`}>
<div className="flex justify-center mb-2">
```

```
<div className={`w-20 h-20 rounded-2xl bg-[#3A7070]/10 flex items-center justify-center
text-
```
[#3A7070]`}><Shield size={40} />

```
</div>
<div className="text-center">
<h2 className="text-3xl font-black text-slate-800 mb-3">Clinical Hub</h2>
<p className="text-slate-500 font-medium text-sm">Strictly for authorized medical
personnel.</p>
</div>
<div className="space-y-5 mt-2">
<div>
<label className="block text-sm font-bold text-slate-700 mb-2">Doctor ID / Email</label>
<input type="text" placeholder="doctor@keffi.ai" className={`w-full p-4 rounded-xl bg-
slate-50
```
border border-slate-200 outline-none text-slate-800 font-medium text-base focus:border-[#3A7070] transition-

all`} />

```
</div>
<div>
<label className="block text-sm font-bold text-slate-700 mb-2">Secure Passcode</label>
<input type="password" placeholder="••••••••" className={`w-full p-4 rounded-xl bg-slate-
50 border
```
border-slate-200 outline-none text-slate-800 font-medium text-base focus:border-[#3A7070] transition-all`} />

```
</div>
</div>
<div className="pt-4 space-y-4">
<button onClick={() => setView('admin-dashboard')} className={`w-full py-4 rounded-xl
font-bold
```
text-base ${theme.btnTeal}`}>Authenticate</button>

```
<button onClick={() => setView('landing')} className="w-full text-center text-slate-400
font-bold text-
```
sm hover:text-slate-600 transition-colors">Back to Home</button>

```
</div>
</div>
</div>
```
);

};

// ==========

// 4. FUNCTIONAL PATIENT COMPONENTS

// ==========

```
const DailyMoodCheckIn = ({ patientId, onComplete }) => {
```

```
const [isSubmitting, setIsSubmitting] = useState(false);
const moods = [
```
{ score: 1, label: 'Heavy', emoji: ' ', color: 'text-slate-600', hover: 'hover:bg-slate-100 hover:border-slate-

300' },

{ score: 2, label: 'Anxious', emoji: ' ', color: 'text-orange-500', hover: 'hover:bg-orange-50 hover:border-

orange-200' },

{ score: 3, label: 'Numb', emoji: ' ', color: 'text-slate-500', hover: 'hover:bg-slate-50 hover:border-slate-300'

},

{ score: 4, label: 'Okay', emoji: ' ', color: 'text-teal-600', hover: 'hover:bg-teal-50 hover:border-teal-200' },

{ score: 5, label: 'Calm', emoji: ' ', color: 'text-green-600', hover: 'hover:bg-green-50 hover:border-green-

200' }

];

```
const handleMoodSelect = async (mood) => {
```
setIsSubmitting(true);

try {

```
const response = await fetch('http://localhost:8000/api/patient/check-in', {
```
method: 'POST',

headers: { 'Content-Type': 'application/json' },

body: JSON.stringify({

patient_id: patientId || "P-102",

emoji_score: mood.score,

sentiment_label: mood.label

})

});

await response.json();

onComplete(mood);

} catch (error) {

console.error("Failed to log mood", error);

```
onComplete(mood);
}
};
return (
<div className="flex flex-col items-center justify-center h-full w-full animate-fade-in
px-6 max-w-4xl mx-
auto">
<h2 className="text-3xl font-black text-slate-800 mb-4 text-center">
Welcome to your Sanctuary.
</h2>
<p className="text-lg text-slate-500 mb-12 text-center">
Before we begin, how is your mind feeling today?
</p>
<div className="grid grid-cols-2 md:grid-cols-5 gap-4 md:gap-6 w-full">
{moods.map((m) => (
<button
key={m.score}
disabled={isSubmitting}
onClick={() => handleMoodSelect(m)}
className={`p-6 md:p-8 rounded-[2rem] bg-white border-2 border-slate-100 shadow-sm flex
flex-col
items-center gap-4 transition-all duration-300 hover:-translate-y-1 ${m.hover}`}
>
<span className="text-5xl">{m.emoji}</span>
<span className={`font-bold text-base ${m.color}`}>{m.label}</span>
</button>
))}
</div>
{isSubmitting && <p className="mt-12 text-sm font-bold text-slate-400 animate-
pulse">Syncing with
Keffi...</p>}
</div>
);
};
// HELPER: No longer stripping options so numbered lists stay in text
const parseMessageText = (text) => {
return { mainText: text || '', options: [] };
```

```
};
// 4.1 Enhanced Chat Page
const ChatArea = ({ setGlobalPoints, globalPoints, userData }) => {
const [moodSet, setMoodSet] = useState(false);
const [messages, setMessages] = useState([]);
const [input, setInput] = useState('');
const [isRecording, setIsRecording] = useState(false);
const [showSOS, setShowSOS] = useState(false);
const [isTyping, setIsTyping] = useState(false);
const [showAppointmentPopup, setShowAppointmentPopup] = useState(false);
const [appointmentPrompted, setAppointmentPrompted] = useState(false);
const chatEndRef = useRef(null);
const recognitionRef = useRef(null);
const scrollToBottom = () => {
chatEndRef.current?.scrollIntoView({ behavior: "smooth" });
};
useEffect(() => {
scrollToBottom();
}, [messages]);
useEffect(() => {
if ('webkitSpeechRecognition' in window || 'SpeechRecognition' in window) {
const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
recognitionRef.current = new SpeechRecognition();
recognitionRef.current.continuous = false;
recognitionRef.current.interimResults = false;
recognitionRef.current.lang = 'en-US';
recognitionRef.current.onresult = (event) => {
const transcript = event.results[0][0].transcript;
setInput(prev => prev + (prev ? " " : "") + transcript);
setIsRecording(false);
};
recognitionRef.current.onerror = () => setIsRecording(false);
recognitionRef.current.onend = () => setIsRecording(false);
}
}, []);
const toggleRecording = () => {
```

```
if (isRecording) {
recognitionRef.current?.stop();
setIsRecording(false);
} else {
if(recognitionRef.current) {
recognitionRef.current.start();
setIsRecording(true);
} else {
alert("Voice recognition is not supported in this browser.");
}
}
};
const handleMoodSelect = (mood) => {
setMoodSet(true);
setMessages([
{ id: 1, sender: 'keffi', time: new Date().toLocaleTimeString([], {hour: '2-digit', minute:'2-digit'}), text: `Hi
${userData?.name || 'there'}. I see you're feeling a bit ${mood.label.toLowerCase()} today. I'm here for you. Do
you want to talk about it?` }
]);
};
const handleSend = async (textOverride) => {
const textToSend = textOverride || input;
if (!textToSend.trim()) return;
const newMsg = { id: Date.now(), sender: 'user', time: new Date().toLocaleTimeString([], {hour: '2-digit',
minute:'2-digit'}), text: textToSend };
setMessages(prev => [...prev, newMsg]);
setInput('');
setGlobalPoints(p => p + 10);
setIsTyping(true);
```

```
try {
const response = await axios.post('http://localhost:8000/api/chat', {
message: textToSend,
patient_id: userData?.patient_id || "P-102"
});
const botResponse = response.data.reply || "I'm here for you.";
setIsTyping(false);
setMessages(prev => [...prev, {
id: Date.now() + 1,
sender: 'keffi',
time: new Date().toLocaleTimeString([], {hour: '2-digit', minute:'2-digit'}),
text: botResponse
}]);
if (response.data.requires_appointment && !appointmentPrompted) {
setAppointmentPrompted(true);
setTimeout(() => setShowAppointmentPopup(true), 1500);
}
if ('speechSynthesis' in window) {
window.speechSynthesis.cancel();
const utterance = new SpeechSynthesisUtterance(botResponse.replace(/[#*]/g, ''));
utterance.lang = 'en-US';
utterance.pitch = 0.9;
utterance.rate = 0.85;
window.speechSynthesis.speak(utterance);
}
} catch (err) {
console.error(err);
setIsTyping(false);
setMessages(prev => [...prev, {
id: Date.now() + 1,
sender: 'keffi',
```

```
time: new Date().toLocaleTimeString([], {hour: '2-digit', minute:'2-digit'}),
text: "I'm having a hard time reaching my thoughts right now (Backend Offline). Let's take a deep breath
together."
}]);
}
};
const handleBookAppointment = async () => {
setShowAppointmentPopup(false);
try {
await axios.post('http://localhost:8000/api/book_appointment', {
patient_id: "P-102",
name: userData?.name || "Patient",
phone: userData?.phone || "9876543210",
email: userData?.email || "patient@keffi.ai"
});
alert("Appointment automation triggered successfully! You will receive a WhatsApp message shortly.");
} catch (err) {
console.error("Failed to book appointment", err);
alert("Failed to trigger appointment automation.");
}
};
if (!moodSet) {
return <DailyMoodCheckIn patientId="P-102" onComplete={handleMoodSelect} />;
}
return (
<div className="grid grid-rows-[auto_1fr_auto] h-full w-full relative animate-fade-in mx-auto overflow-
hidden">
<div className="flex justify-between items-center px-6 md:px-8 pt-6 pb-4 z-20 border-b border-white/40
bg-white/30 backdrop-blur-sm">
```

```
<div className="flex items-center gap-4">
<div className="w-12 h-12 bg-white rounded-full flex items-center justify-center shadow-
sm">
<KeffiLogo size="w-7 h-7" />
</div>
<div>
<h2 className="text-xl font-bold text-slate-800">Keffi</h2>
<div className="flex items-center gap-2 text-xs text-[#8FA989] font-medium">
<div className="w-2 h-2 rounded-full bg-[#8FA989] animate-pulse"></div> Active &
Listening
</div>
</div>
</div>
<div className="flex items-center gap-3">
<button onClick={() => setShowAppointmentPopup(true)} className="px-4 py-2 rounded-full
bg-slate-
```
50 text-slate-600 font-bold text-xs hover:bg-slate-100 flex items-center gap-2 transition-colors">
```
<User size={14}/> Therapist
</button>
<button onClick={() => setShowSOS(true)} className="px-4 py-2 rounded-full bg-red-50
text-red-600
```
font-bold text-xs hover:bg-red-100 flex items-center gap-2 transition-colors">
```
<PhoneCall size={14}/> SOS
</button>
</div>
</div>
```
{showSOS && (
```
<div className="absolute top-24 left-1/2 transform -translate-x-1/2 z-50 bg-white p-8
rounded-[2rem]
```
shadow-2xl border border-red-100 flex flex-col items-center animate-fade-in w-80 text-center">
```
<div className="w-16 h-16 bg-red-50 rounded-2xl flex items-center justify-center text-
red-500 mb-
```
6"><AlertTriangle size={32}/>
```
<h3 className="text-xl font-black text-slate-800 mb-2">Emergency Hotline</h3>
<p className="text-sm text-slate-500 mb-6 font-medium">You are not alone. Please call
iCall India for
```
immediate support.</p>
```
<div className="text-2xl font-black text-[#2C5555] mb-6 tracking-widest">9152987821</div>
<button onClick={() => setShowSOS(false)} className="px-8 py-3 rounded-xl bg-slate-100
text-slate-
```
600 font-bold text-sm hover:bg-slate-200 w-full transition-colors">Close</button>
```
</div>
```
)}

{showAppointmentPopup && (
```
<div className="absolute top-24 left-1/2 transform -translate-x-1/2 z-[60] bg-white p-8
rounded-[2rem]
```

shadow-2xl border border-slate-100 flex flex-col items-center animate-fade-in w-80 text-center">

```
<div className="w-16 h-16 bg-slate-50 rounded-2xl flex items-center justify-center text-[#3A7070] mb-
```

6 text-3xl">

```
<h3 className="text-xl font-black text-slate-800 mb-2">You're not alone</h3>
<p className="text-sm text-slate-500 mb-8 font-medium">We noticed you're going through a tough
```

time. Would you like to schedule an automatic appointment with a human therapist?</p>

```
<div className="flex flex-col gap-3 w-full">
<button onClick={handleBookAppointment} className={`w-full py-3.5 rounded-xl font-bold text-sm
```

${theme.btnTeal}`}>Yes, Book Session</button>

```
<button onClick={() => setShowAppointmentPopup(false)} className={`w-full py-3.5 rounded-xl
```

font-bold text-sm bg-slate-50 text-slate-500 hover:bg-slate-100 transition-colors`}>Not Right Now</button>

```
</div>
</div>
```

)}

```
<div className="overflow-y-auto p-6 md:p-8 flex flex-col gap-6 z-10 min-h-0 relative">
```

{messages.map(m => {

```
const { mainText, options } = m.sender === 'keffi' ? parseMessageText(m.text) : { mainText: m.text,
```

options: [] };

```
return (
<div key={m.id} className={`flex w-full ${m.sender === 'user' ? 'justify-end' : 'justify-start'} animate-
```

fade-in-up`}>

```
<div className={`flex flex-col ${m.sender === 'user' ? 'items-end' : 'items-start'} max-w-[85%]`}>
<div className={`whitespace-pre-wrap p-4 md:p-5 text-sm md:text-base font-medium leading-
```

relaxed ${

m.sender === 'user'

? 'bg-white text-slate-800 rounded-[1.5rem] rounded-tr-sm shadow-sm border border-slate-100'

: 'bg-[#3A7070]/10 text-slate-800 rounded-[1.5rem] rounded-tl-sm border border-[#3A7070]/10'

}`}>

{mainText}

```
</div>
```

key={idx}

```
onClick={() => handleSend(cleanOpt)}
className={`flex-1 min-w-[120px] text-left p-3 rounded-xl bg-white border border-slate-200
shadow-sm hover:border-[#3A7070]/50 hover:bg-[#3A7070]/5 transition-colors flex items-start
gap-2`}
>
<div className="w-5 h-5 shrink-0 rounded-full bg-[#3A7070]/10 text-[#3A7070] flex items-
center justify-center font-bold text-[10px] mt-0.5">
{idx + 1}
</div>
<span className="text-sm font-bold text-slate-700">
{cleanOpt}
</span>
</button>
);
})}
</div>
)}
<span className={`text-[10px] text-slate-400 font-bold uppercase tracking-wider mt-2 px-
1`}>
{m.time}
</span>
</div>
</div>
);
})}
{isTyping && (
<div className="flex flex-col self-start items-start max-w-[85%]">
<div className="p-4 rounded-[1.5rem] bg-[#3A7070]/10 flex gap-1.5 items-center rounded-
tl-sm">
<div className="w-2 h-2 rounded-full bg-[#3A7070] animate-bounce" style={{
animationDelay:
'0ms' }}></div>
<div className="w-2 h-2 rounded-full bg-[#3A7070] animate-bounce" style={{
animationDelay:
'150ms' }}></div>
<div className="w-2 h-2 rounded-full bg-[#3A7070] animate-bounce" style={{
animationDelay:
'300ms' }}></div>
</div>
```

```
</div>
)}
<div ref={chatEndRef} className="h-4" />
</div>
<div className="p-4 md:p-6 bg-white/40 backdrop-blur-sm border-t border-white/40 z-20">
<div className="flex gap-2 mb-3 overflow-x-auto pb-2 scrollbar-hide">
```
{["I feel anxious", "Can't sleep", "Just want to vent"].map((qr, i) => (
```
<button key={i} onClick={() => handleSend(qr)} className="whitespace-nowrap px-4 py-2
rounded-
```
full text-xs font-bold bg-white border border-slate-200 text-slate-600 hover:bg-slate-50
transition-colors

shadow-sm hover:shadow">

{qr}

```
</button>
```
))}
```
</div>
<div className="flex gap-2 relative">
<button onClick={toggleRecording} className={`p-4 rounded-2xl ${isRecording ? 'bg-red-50
text-red-
```
500 animate-pulse' : 'bg-white text-slate-400 hover:text-[#3A7070] hover:bg-slate-50'} border
border-slate-200

shadow-sm transition-colors shrink-0`}>

```
<Mic size={20} />
</button>
<input
```
value={input} onChange={(e) => setInput(e.target.value)} onKeyPress={(e) => e.key ===
'Enter' &&

handleSend()}

placeholder={isRecording ? "Listening..." : "Type your feelings safely here..."}

className="flex-1 rounded-2xl border border-slate-200 px-5 py-4 text-sm outline-none
focus:border-

[#3A7070] focus:ring-2 focus:ring-[#3A7070]/20 shadow-sm transition-all"

/>

```
<button onClick={() => handleSend()} className="px-6 rounded-2xl bg-[#3A7070] hover:bg-
```
[#2C5555] text-white flex items-center justify-center transition-colors shadow-sm shrink-0">

```
<Send size={18} />
</button>
</div>
<div className="text-center text-[10px] text-slate-400 font-bold uppercase tracking-
widest mt-4">
```

Keffi AI is not a substitute for medical diagnosis.

```
</div>
</div>
</div>
);

};

// 4.2 Peace Log

const PeaceLog = () => (
<div className="h-full flex flex-col max-w-5xl mx-auto w-full">
<h2 className="text-2xl font-black text-slate-800 mb-8">Peace Log</h2>
<div className="grid grid-cols-1 md:grid-cols-2 gap-6 overflow-y-auto pb-8 pr-2">
{[

{ date: 'Today, 10:00 AM', mood: 'Anxious', title: 'Morning Panic', desc: 'Discussed work pressure and did a

quick 4-7-8 breathing session.' },

{ date: 'Yesterday, 9:00 PM', mood: 'Calm', title: 'Night Reflections', desc: 'Used the gratitude jar. Felt

significantly calmer before bed.' },

{ date: '25 April 2026', mood: 'Heavy', title: 'Trauma Processing', desc: 'Keffi guided through severe

anxiety. Grounding techniques used.' },

{ date: '22 April 2026', mood: 'Stressed', title: 'Work Stress', desc: 'Vented about the upcoming presentation.

Keffi helped reframe thoughts.' }

].map((log, i) => (

<div key={i} className={`p-8 rounded-[2rem] ${theme.outset} cursor-pointer
${theme.outsetHover} flex
flex-col`}>

<div className="flex justify-between items-center mb-5">
<span className="text-xs text-[#8FA989] font-bold uppercase tracking-
widest">{log.date}</span>
<span className="px-3 py-1 rounded-md bg-slate-50 border border-slate-100 text-xs font-
bold text-
slate-500">{log.mood}</span>

</div>
<h3 className="text-lg font-bold text-slate-800 mb-3">{log.title}</h3>
<p className="text-slate-500 leading-relaxed text-sm flex-1">{log.desc}</p>
<div className="mt-6 flex items-center text-[#3A7070] font-bold text-sm group">
Read full log <ArrowRight size={16} className="ml-1 group-hover:translate-x-1 transition-
```

transform"/>

```
</div>
</div>
```
))}

```
</div>
</div>
```
);

// 4.3 My Journey

```
const MyJourney = () => (
<div className="h-full flex flex-col max-w-5xl mx-auto w-full">
<h2 className="text-2xl font-black text-slate-800 mb-8">Emotional Landscape</h2>
<div className={`w-full h-80 rounded-[2.5rem] bg-slate-50 border border-slate-100 mb-8
relative overflow-
```
hidden flex items-end justify-center shadow-inner`}>

```
<div className="absolute top-8 right-12 w-24 h-24 rounded-full bg-gradient-to-tr from-
[#D4A373] to-
```
white blur-md shadow-[0_0_40px_#D4A373]">

```
<div className="w-[120%] h-40 bg-[#8FA989] rounded-[100%] absolute -bottom-8 opacity-
40"></div>
<div className="w-[80%] h-48 bg-[#3A7070] rounded-[100%] absolute -bottom-10 opacity-70
left-[-
```
10%]">

```
<div className="w-[90%] h-44 bg-[#548a8a] rounded-[100%] absolute -bottom-8 opacity-80
right-[-
```
10%]">

```
<div className={`absolute top-8 left-8 px-6 py-3 rounded-2xl bg-white/80 backdrop-blur-md
border
```
border-white shadow-sm`}>

```
<h3 className="text-lg font-bold text-slate-800">You are Thriving  </h3>
</div>
</div>
<div className="grid grid-cols-1 md:grid-cols-3 gap-6">
<div className={`p-8 rounded-[2rem] bg-white border border-slate-100 shadow-sm flex flex-
col items-
```
center justify-center gap-3`}>

```
<div className="text-4xl font-black text-[#3A7070]">12</div>
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest">Day
Streak</div>
</div>
<div className={`p-8 rounded-[2rem] bg-white border border-slate-100 shadow-sm flex flex-
col items-
```
center justify-center gap-3`}>

```
<div className="text-4xl font-black text-[#8FA989]">85%</div>
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest">Calm
Status</div>
```

```
</div>
<div className={`p-8 rounded-[2rem] bg-white border border-slate-100 shadow-sm flex flex-col items-
```
center justify-center gap-3`}>
```
<div className="text-4xl font-black text-[#D4A373]">4</div>
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest">Tools Used</div>
</div>
</div>
</div>
```
);

// 4.4 Mind Tools

```
const MindTools = () => {
const [activeTool, setActiveTool] = useState(null);
const [worryText, setWorryText] = useState('');
const [isBurning, setIsBurning] = useState(false);
const [gratitudeText, setGratitudeText] = useState('');
const [gratitudeList, setGratitudeList] = useState([]);
const [groundingStep, setGroundingStep] = useState(0);
const handleBurn = () => {
```
setIsBurning(true);

setTimeout(() => { setWorryText(''); setIsBurning(false); setActiveTool(null); }, 2000);

};

if (activeTool === 'breathing') {

```
return (
<div className="h-full flex flex-col items-center justify-center space-y-12 animate-fade-in">
<div className="text-center">
<h2 className="text-3xl font-black text-slate-800 mb-4">4-7-8 Breathing</h2>
<p className="text-slate-500 text-base">Follow the circle to reduce anxiety.</p>
</div>
<div className="relative w-80 h-80 flex items-center justify-center">
<div className="absolute inset-0 bg-[#3A7070] rounded-full opacity-10 animate-ping"
```
style={{animationDuration: '4s'}}>

```
<div className={`w-48 h-48 rounded-full bg-white shadow-xl border border-slate-100 flex items-center
```
justify-center text-[#3A7070] font-black text-xl z-10`}>Breathe In

```
</div>
<button onClick={() => setActiveTool(null)} className={`px-8 py-3 rounded-xl font-bold text-base
```
${theme.btnOutline}`}>Stop & Go Back</button>

```
</div>
```
);

}

```
if (activeTool === 'worry') {
return (
<div className="h-full flex flex-col items-center justify-center max-w-2xl mx-auto space-y-8 w-full
animate-fade-in">
<div className="text-center">
<h2 className="text-3xl font-black text-[#D4A373] mb-4">Worry Burner</h2>
<p className="text-slate-500 text-base">Type what's bothering you, and let it go into the ash.</p>
</div>
<textarea
value={worryText} onChange={(e) => setWorryText(e.target.value)}
className={`w-full h-64 p-8 rounded-[2rem] bg-white border border-slate-200 shadow-inner outline-
none text-slate-800 text-base resize-none transition-all duration-1000 ${isBurning ? 'blur-xl opacity-0 scale-110'
: ''}`}
placeholder="I am worried about..."
/>
<div className="flex gap-4 w-full">
<button onClick={() => setActiveTool(null)} className={`flex-1 py-4 rounded-xl font-bold text-base
${theme.btnOutline}`}>Cancel</button>
<button onClick={handleBurn} className={`flex-1 py-4 rounded-xl font-bold text-base bg-[#D4A373]
text-white shadow-lg hover:-translate-y-1 transition-transform`}>Burn Worry</button>
</div>
</div>
);
}
if (activeTool === 'gratitude') {
return (
<div className="h-full flex flex-col items-center max-w-3xl mx-auto space-y-8 w-full animate-fade-in p-
6">
<div className="text-center">
<h2 className="text-3xl font-black text-[#8FA989] mb-4">Gratitude Jar</h2>
<p className="text-slate-500 text-base">Drop small moments of joy here.</p>
</div>
<div className="flex w-full gap-4">
<input
```

```
value={gratitudeText} onChange={e => setGratitudeText(e.target.value)}
placeholder="I am grateful for..."
className={`flex-1 p-4 rounded-2xl bg-white border border-slate-200 shadow-sm outline-none text-
slate-800 font-medium text-base`}
onKeyPress={e => {
if(e.key === 'Enter' && gratitudeText) {
setGratitudeList([{id: Date.now(), text: gratitudeText}, ...gratitudeList]);
setGratitudeText('');
}
}}
/>
<button onClick={() => {
if(gratitudeText) {
setGratitudeList([{id: Date.now(), text: gratitudeText}, ...gratitudeList]);
setGratitudeText('');
}
}} className={`px-8 py-4 rounded-2xl bg-[#8FA989] text-white font-bold shadow-md hover:-translate-
y-1 transition-transform text-base`}>Drop</button>
</div>
<div className={`flex-1 w-full rounded-[2.5rem] bg-slate-50 border-8 border-[#8FA989]/10 p-8 flex flex-
col-reverse items-center justify-start overflow-y-auto relative`}>
<div className="w-48 h-8 rounded-[100%] bg-slate-200 absolute -top-4 opacity-50 blur-md"></div>
{gratitudeList.length === 0 && <div className="text-slate-400 font-bold text-sm absolute top-
1/2">Your jar is empty.</div>}
{gratitudeList.map(g => (
<div key={g.id} className="bg-gradient-to-r from-[#8FA989] to-[#649e9e] text-white px-6 py-3.5
rounded-full mb-3 shadow-lg transform rotate-[-2deg] font-bold text-sm animate-fade-in">
{g.text}
</div>
```

))}

```
</div>
<button onClick={() => setActiveTool(null)} className={`w-full py-4 rounded-xl font-bold
text-base
```

${theme.btnOutline}`}>Back to Tools</button>

```
</div>
```

);

}

if (activeTool === 'grounding') {

```
const steps = [
```

{ num: 5, text: "Things you can SEE", color: "text-[#3A7070]", bg: "bg-[#3A7070]" },

{ num: 4, text: "Things you can FEEL", color: "text-[#D4A373]", bg: "bg-[#D4A373]" },

{ num: 3, text: "Things you can HEAR", color: "text-[#8FA989]", bg: "bg-[#8FA989]" },

{ num: 2, text: "Things you can SMELL", color: "text-slate-600", bg: "bg-slate-600" },

{ num: 1, text: "Thing you can TASTE", color: "text-[#3A7070]", bg: "bg-[#3A7070]" },

];

```
const current = steps[groundingStep];
return (
<div className="h-full flex flex-col items-center justify-center max-w-3xl mx-auto w-full
animate-fade-
```

in">

```
<div className="text-center mb-12">
<h2 className="text-3xl font-black text-slate-800 mb-4">5-4-3-2-1 Grounding</h2>
<p className="text-slate-500 text-base">Halt panic and come back to the present.</p>
</div>
```

{groundingStep < 5 ? (

```
<div className={`p-16 rounded-[3rem] bg-white border border-slate-100 shadow-xl flex
flex-col items-
```

center text-center w-full`}>

```
<div className={`text-7xl font-black ${current.color} mb-6`}>{current.num}</div>
<h3 className={`text-2xl font-black text-slate-800 mb-10 uppercase tracking-
```

widest`}>{current.text}</h3>

```
<p className="text-slate-500 font-medium text-base mb-10">Take your time. Look around
you. Name
```

them silently or out loud.</p>

```
<button onClick={() => setGroundingStep(s => s + 1)} className={`w-full py-4 rounded-xl
text-white
```

font-bold text-base ${current.bg} shadow-lg hover:-translate-y-1 transition-transform`}>Next Step</button>

```
</div>
) : (
<div className={`p-16 rounded-[3rem] bg-white border border-slate-100 shadow-xl flex flex-col items-
center text-center w-full`}>
<div className={`w-24 h-24 rounded-2xl bg-[#8FA989]/10 text-[#8FA989] flex items-center justify-
center mb-8`}><Star size={48}/></div>
<h3 className="text-2xl font-black text-slate-800 mb-6">You did great.</h3>
<p className="text-slate-500 font-medium text-base mb-10">Welcome back to the present
moment.</p>
<button onClick={() => {setGroundingStep(0); setActiveTool(null);}} className={`w-full py-4
rounded-xl font-bold text-base ${theme.btnOutline}`}>Finish</button>
</div>
)}
</div>
);
}
const allTools = [
{ id: 'breathing', title: '4-7-8 Breathing', desc: 'Reduce heart rate', icon: Activity, color: 'text-[#3A7070]',
active: true },
{ id: 'worry', title: 'Worry Burner', desc: 'Release anxiety visually', icon: Sparkles, color: 'text-[#D4A373]',
active: true },
{ id: 'gratitude', title: 'Gratitude Jar', desc: 'Shift perspective', icon: Heart, color: 'text-[#8FA989]', active: true
},
{ id: 'grounding', title: '5-4-3-2-1 Grounding', desc: 'Halt panic attacks', icon: Star, color: 'text-[#3A7070]',
active: true },
{ id: 'bodyscan', title: 'Body Scan', desc: 'Release physical tension', icon: User, color: 'text-[#D4A373]', active:
false },
{ id: 'moodtracker', title: 'Mood Tracker', desc: 'Identify daily patterns', icon: PieChart, color: 'text-
```

[#8FA989]', active: false },

{ id: 'reframing', title: 'Cognitive Reframing', desc: 'Challenge negative thoughts', icon: MessageCircle, color:

'text-[#3A7070]', active: false },

{ id: 'thermometer', title: 'Anxiety Thermometer', desc: 'Measure distress', icon: AlertTriangle, color: 'text-

[#D4A373]', active: false },

{ id: 'sleep', title: 'Sleep Wind-down', desc: 'Prepare for deep rest', icon: Smile, color: 'text-[#8FA989]', active:

false },

{ id: 'journal', title: 'Guided Journal', desc: 'Unlocks at Level 2', icon: BookOpen, color: 'text-slate-400',

active: false, locked: true },

];

```
return (
<div className="h-full flex flex-col max-w-7xl mx-auto w-full">
<h2 className="text-2xl font-black text-slate-800 mb-8">Mind Tools Sandbox</h2>
<div className="grid grid-cols-1 md:grid-cols-3 lg:grid-cols-4 gap-6 overflow-y-auto pb-8
pr-2">
{allTools.map((tool) => (
<div key={tool.id} onClick={() => tool.active ? setActiveTool(tool.id) : null}
className={`p-6 rounded-[2rem] bg-white border border-slate-100 shadow-sm flex flex-col
items-
center justify-center gap-4 text-center ${tool.active ? theme.outsetHover : 'opacity-60'}
${tool.locked ? 'cursor-
not-allowed opacity-40' : 'cursor-pointer'}`}>
<div className={`w-14 h-14 rounded-2xl bg-slate-50 flex items-center justify-center
${tool.color}`}><tool.icon size={24} /></div>
<div>
<h3 className="font-bold text-slate-800 text-base mb-1">{tool.title}</h3>
<p className="text-xs text-slate-500 font-medium">{tool.desc}</p>
</div>
</div>
))}
</div>
</div>
);
```

```
};

// 4.5 Rewards
const Rewards = ({ points }) => (
<div className="h-full flex flex-col items-center justify-center max-w-2xl mx-auto w-full">
<div className={`w-full p-10 rounded-[2.5rem] bg-white shadow-sm border border-slate-100 flex flex-col
items-center text-center mb-10`}>
<Gift size={48} className="text-[#D4A373] mb-6" />
<h2 className="text-sm font-bold text-slate-400 uppercase tracking-widest mb-4">Total Keffi Points</h2>
<div className="text-5xl font-black text-[#3A7070]">{points}</div>
</div>
<div className="w-full space-y-4">
<h3 className="font-black text-slate-800 text-xl mb-4">Unlock Goals</h3>
<div className={`p-5 rounded-2xl bg-slate-50 border border-slate-100 flex justify-between items-center
opacity-60`}>
<span className="font-bold text-slate-600 text-base">Mindfulness Badge</span>
<span className="font-bold text-[#3A7070] text-base">100 pts (Unlocked)</span>
</div>
<div className={`p-5 rounded-2xl bg-white border border-slate-100 shadow-sm flex justify-between items-
center`}>
<span className="font-bold text-slate-800 text-base">Free Therapist Session</span>
<span className="font-bold text-[#D4A373] text-base">1000 pts</span>
</div>
<div className="w-full bg-slate-100 rounded-full h-3 mt-4 overflow-hidden border border-slate-200">
<div className="bg-[#3A7070] h-3 transition-all duration-1000" style={{width:
`${Math.min((points/1000)*100, 100)}%`}}></div>
</div>
{points >= 1000 ? (
<button className={`w-full py-4 mt-6 rounded-xl font-bold text-base
${theme.btnTeal}`}>Claim
Therapist Session</button>
) : (
<button disabled className={`w-full py-4 mt-6 rounded-xl font-bold text-base bg-slate-100 text-slate-400
cursor-not-allowed`}>Chat more to Unlock</button>
)}
</div>
</div>
);
```

// 4.6 Friends

```
const FriendsSync = () => (
<div className="h-full flex flex-col items-center justify-center">
<div className="text-center mb-10">
<h2 className="text-2xl font-black text-slate-800 mb-3">Neighbor Sync</h2>
<p className="text-slate-500 text-base">You are not alone. See the abstract mood of
people near you.</p>
</div>
<div className={`w-80 h-80 rounded-full bg-slate-50 border border-slate-100 relative flex
items-center
justify-center shadow-inner`}>
<div className="absolute w-64 h-64 rounded-full border border-slate-200 opacity-
70"></div>
<div className="absolute w-40 h-40 rounded-full border border-slate-200 opacity-
70"></div>
<div className={`w-10 h-10 bg-[#3A7070] rounded-full z-10 shadow-lg animate-
pulse`}></div>
<div className="absolute mt-16 font-bold text-xs text-[#3A7070] bg-white/80 px-3 py-1
rounded-full
border border-slate-100 shadow-sm">You</div>
<div className="absolute top-16 left-16 w-8 h-8 bg-[#8FA989] rounded-full shadow-
[0_0_20px_#8FA989]
cursor-pointer hover:scale-125 transition-transform" title="Calm Neighbor"></div>
<div className="absolute bottom-20 right-16 w-8 h-8 bg-[#D4A373] rounded-full shadow-
[0_0_20px_#D4A373] cursor-pointer hover:scale-125 transition-transform" title="Anxious
Neighbor"></div>
<div className="absolute top-24 right-10 w-8 h-8 bg-[#8FA989] rounded-full shadow-
[0_0_20px_#8FA989] cursor-pointer hover:scale-125 transition-transform" title="Calm
Neighbor"></div>
</div>
<div className={`mt-12 p-6 rounded-2xl bg-white border border-slate-100 shadow-sm flex
items-center
gap-5`}>
<Heart className="text-[#D4A373]" size={24} />
<span className="font-bold text-slate-800 text-base">Send a Virtual Hug</span>
<button className={`px-5 py-2.5 rounded-xl text-sm font-bold ${theme.btnOutline}`}>Send
Love</button>
</div>
</div>
);
// 4.7 Profile
const ProfileVault = ({ userData }) => (
<div className="h-full flex flex-col items-center justify-center">
<div className={`max-w-2xl w-full p-10 rounded-[2.5rem] bg-white border border-slate-100
shadow-sm`}>
```

```
<div className="flex flex-col items-center mb-8">
<div className={`w-24 h-24 rounded-3xl bg-slate-50 flex items-center justify-center text-
[#3A7070] mb-
5`}>
<User size={40} />
</div>
<h2 className="text-2xl font-black text-slate-800">Identity Vault</h2>
</div>
<div className="grid grid-cols-1 md:grid-cols-2 gap-6">
<div className={`p-5 rounded-2xl bg-slate-50 border border-slate-100`}>
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest mb-2">Full
Name</div>
<div className="text-lg font-bold text-slate-800">{userData?.name || 'Guest User'}</div>
</div>
<div className={`p-5 rounded-2xl bg-slate-50 border border-slate-100`}>
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest mb-2">Age /
DOB /
Gender</div>
<div className="text-lg font-bold text-slate-800">{userData?.age || '25'} •
{userData?.dob || 'Not Set'} •
{userData?.gender || 'Not Set'}</div>
</div>
<div className={`p-5 rounded-2xl bg-slate-50 border border-slate-100`}>
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest mb-2">Phone /
Email</div>
<div className="text-lg font-bold text-slate-800">{userData?.phone || '+91 00000000'}
<br/><span
className="text-sm font-medium text-slate-500 mt-1 inline-block">{userData?.email ||
'email@example.com'}</span></div>
</div>
<div className={`p-5 rounded-2xl bg-slate-50 border border-slate-100`}>
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest mb-
2">Sanctuary
Location</div>
<div className="text-lg font-bold text-slate-800 flex items-center gap-2"><MapPin
size={20}
className="text-[#3A7070]"/> {userData?.place || 'Salem, TN'}</div>
</div>
</div>
</div>
</div>
);

// ==========================================

// 5. MASTER PATIENT DASHBOARD
```

```
// ==========================================
const PatientDashboard = ({ setView, userData }) => {
const [activePage, setActivePage] = useState('chat');
const [globalPoints, setGlobalPoints] = useState(0);
const [isSidebarOpen, setIsSidebarOpen] = useState(true);
const menuItems = [
{ id: 'chat', label: 'Chat with Keffi', icon: MessageCircle },
{ id: 'history', label: 'Peace Log', icon: BookOpen },
{ id: 'journey', label: 'My Journey', icon: TrendingUp },
{ id: 'tools', label: 'Mind Tools', icon: Sparkles },
{ id: 'rewards', label: 'Rewards', icon: Gift },
{ id: 'friends', label: 'Neighbor Sync', icon: Users },
{ id: 'account', label: 'My Account', icon: Shield },
];
const renderContent = () => {
switch(activePage) {
case 'chat': return <ChatArea setGlobalPoints={setGlobalPoints} globalPoints={globalPoints}
userData={userData} />;
case 'history': return <PeaceLog />;
case 'journey': return <MyJourney />;
case 'tools': return <MindTools />;
case 'rewards': return <Rewards points={globalPoints} />;
case 'friends': return <FriendsSync />;
case 'account': return <ProfileVault userData={userData} />;
default: return <ChatArea setGlobalPoints={setGlobalPoints} globalPoints={globalPoints}
userData={userData} />;
}
};
return (
<div className="flex h-screen w-screen bg-[#f4f7f6] overflow-hidden font-inter text-
slate-800">
{/* Sleek Collapsible Sidebar */}
<div className={`${isSidebarOpen ? 'w-64 md:w-72' : 'w-0 opacity-0'} transition-all
duration-300 h-full
```

bg-white border-r border-slate-200 flex flex-col shrink-0 overflow-hidden relative z-20`}>

{/* Light Green Animated Background */}

```
<div className="absolute -top-[10%] -left-[10%] w-[120%] h-[50%] bg-[#8FA989] rounded-
full mix-
```

blend-multiply blur-[80px] opacity-[0.15] animate-pulse pointer-events-none z-0" style={{animationDuration:

'4s'}}>

```
<div className="absolute -bottom-[10%] -right-[10%] w-[120%] h-[50%] bg-[#EAF4F0]
rounded-full
```

mix-blend-multiply blur-[80px] opacity-70 pointer-events-none z-0" style={{animation: 'pulse 8s infinite

alternate'}}>

{isSidebarOpen && (

```
<div className="flex flex-col h-full w-full p-6 relative z-10">
<div className="flex items-center gap-3 cursor-pointer mb-8" onClick={() =>
setView('landing')}>
<KeffiLogo size="w-8 h-8" />
<h1 className="text-2xl font-black text-[#2C5555] tracking-tight">Keffi AI</h1>
</div>
<div className="flex-1 space-y-2 overflow-y-auto scrollbar-hide">
```

{menuItems.map(item => {

```
const IconComponent = item.icon;
const isActive = activePage === item.id;
return (
<button
```

key={item.id}

onClick={() => setActivePage(item.id)}

className={`w-full flex items-center gap-4 px-5 py-4 rounded-[1.2rem] font-bold text-sm

transition-all duration-300 ${

isActive

? 'bg-[#3A7070] text-white shadow-md shadow-[#3A7070]/20'

: 'text-slate-500 hover:bg-slate-50 hover:text-[#3A7070]'

}`}

>

```
<IconComponent size={18} /> {item.label}
</button>
```

)

})}

```
</div>
<div className="mt-6 px-4 py-4 rounded-[1.2rem] bg-slate-50 border border-slate-100 font-bold text-
```
xs text-[#D4A373] flex items-center justify-center gap-2 shrink-0">

```
<Star size={16} className="fill-[#D4A373] text-[#D4A373] animate-pulse" /> {globalPoints}
```
Sanctuary Points

```
</div>
<button onClick={() => setView('landing')} className="mt-4 text-slate-400 hover:text-slate-600 font-
```
bold text-sm transition-all w-full text-center shrink-0">

Exit Sanctuary

```
</button>
</div>
```
)}

```
</div>
```
{/* Main Content Area */}

```
<div className="flex-1 h-full min-w-0 bg-[#EAF0EE] relative flex flex-col overflow-hidden">
```
{!isSidebarOpen && (

```
<button
```
onClick={() => setIsSidebarOpen(true)}

className="absolute top-6 left-6 z-50 p-2.5 text-slate-500 hover:text-[#3A7070] bg-white border

border-slate-100 shadow-sm rounded-xl hover:-translate-y-1 transition-all"

>

```
<Icon size={20}><line x1="3" y1="12" x2="21" y2="12"/><line x1="3" y1="6" x2="21" y2="6"/><line x1="3" y1="18" x2="21" y2="18"/></Icon>
```

```
</button>
```
)}

{/* Render Active View */}

{renderContent()}

```
</div>
</div>
```
);

};

// ==========

// 6. ENHANCED ADMIN DASHBOARD

```
// ============================================
const AdminDashboard = ({ setView }) => {
const [activeTab, setActiveTab] = useState('overview');
const [selectedPatient, setSelectedPatient] = useState(null);
const [patients, setPatients] = useState([]);
const [inactivePatients, setInactivePatients] = useState([]);
const [analytics, setAnalytics] = useState(null);
useEffect(() => {
const fetchData = async () => {
try {
const [resPat, resInact, resAnalyt] = await Promise.all([
axios.get('http://localhost:8000/api/admin/patients'),
axios.get('http://localhost:8000/api/admin/inactive-patients'),
axios.get('http://localhost:8000/api/admin/analytics')
]);
if (resPat.data && resPat.data.patients) setPatients(resPat.data.patients);
if (resInact.data && resInact.data.patients) setInactivePatients(resInact.data.patients);
if (resAnalyt.data) setAnalytics(resAnalyt.data);
} catch (err) {
console.error("Failed to fetch admin data", err);
}
};
fetchData();
const interval = setInterval(fetchData, 10000);
return () => clearInterval(interval);
}, []);
const adminTabs = [
{ id: 'overview', label: 'System Overview', icon: Activity },
{ id: 'roster', label: 'Patient Roster', icon: Users },
{ id: 'inactive', label: 'Inactive Patients', icon: Frown },
{ id: 'analytics', label: 'NLP Analytics', icon: PieChart },
{ id: 'therapists', label: 'Therapists Allocation', icon: Shield },
{ id: 'settings', label: 'System Settings', icon: Settings },
];
```

```
const handleExportAbstract = async (patientId) => {
try {
const res = await axios.get(`http://localhost:8000/api/patient/${patientId}/report`);
const data = res.data;
const content = `Clinical Abstract for ${data.name || data.patient_id}\n\nMHQ Score:
${data.current_mhq}\nRisk Level: ${data.depression_level}\nAssigned Doctor:
${data.assigned_doctor ||
'Unassigned'}\n\nSummary:\n${data.clinical_abstract}\n`;
const blob = new Blob([content], { type: 'text/plain' });
const url = window.URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = `abstract_${patientId}.txt`;
a.click();
window.URL.revokeObjectURL(url);
} catch(err) {
console.error(err);
alert("Failed to export abstract.");
}
};
const assignTherapist = async (patientId, docName) => {
try {
await axios.post('http://localhost:8000/api/admin/assign-therapist', { patient_id: patientId,
doctor_name:
docName });
alert(`Successfully assigned ${docName} to ${patientId}`);
} catch(err) {
console.error(err);
}
};
return (
<div className={`flex h-screen w-full ${theme.bg} overflow-hidden p-6 gap-6 font-sans
text-slate-800`}>
<div className={`w-72 h-full rounded-[2rem] bg-white border border-slate-100 shadow-sm
flex flex-col p-
6 shrink-0`}>
```

```
<div className="flex flex-col items-center mb-8 pt-4 cursor-pointer" onClick={() =>
setView('landing')}>
<div className={`w-12 h-12 rounded-2xl bg-[#3A7070]/10 flex items-center justify-center
text-
```
[#3A7070] mb-4`}><Shield size={24} />

```
<h1 className="text-xl font-black text-[#2C5555] tracking-tight text-center">Clinical
Hub</h1>
</div>
<div className="flex-1 space-y-2">
```
{adminTabs.map(tab => (

```
<button key={tab.id} onClick={() => {setActiveTab(tab.id); setSelectedPatient(null);}}
```
className={`w-full flex items-center gap-3 px-4 py-3 rounded-xl font-bold text-sm transition-
all

duration-300 ${activeTab === tab.id ? `bg-slate-50 border border-slate-100 text-[#3A7070]
shadow-sm` : `text-

slate-500 hover:text-[#3A7070] hover:bg-slate-50`}`}>

```
<tab.icon size={18} /> {tab.label}
</button>
```
))}

```
</div>
<button onClick={() => setView('landing')} className="mt-auto font-bold text-sm text-
slate-400
```
hover:text-red-500 hover:bg-red-50 p-4 rounded-xl transition-all">Logout</button>

```
</div>
<div className="flex-1 h-full relative">
<div className={`absolute inset-0 rounded-[2rem] bg-[#FDFCF8] border border-slate-100
shadow-inner
```
p-8 overflow-hidden`}>

{selectedPatient ? (

```
<div className={`h-full flex flex-col animate-fade-in`}>
<div className="flex justify-between items-center mb-6">
<div className="flex items-center gap-4">
<button onClick={() => setSelectedPatient(null)} className={`p-2.5 rounded-full bg-white
border
```
border-slate-200 shadow-sm text-slate-500 hover:bg-slate-50`}><ArrowRight
className="rotate-180"

size={18}/></button>

```
<h2 className="text-xl font-black text-slate-800">Patient: {selectedPatient.id}</h2>
</div>
<div className="flex items-center gap-3">
<div className="px-4 py-1.5 rounded-xl bg-white border border-slate-100 shadow-sm font-
bold
```
text-xs text-slate-600 flex items-center gap-2">

Current Chat Category: <span className={

selectedPatient.category?.includes("Depression") || selectedPatient.category?.includes("Panic") ||

selectedPatient.category?.includes("Grief") ? "text-red-500" :

selectedPatient.category?.includes("Anxiety") ||
selectedPatient.category?.includes("Exhaustion")

|| selectedPatient.category?.includes("Burnout") ? "text-orange-500" :

"text-emerald-500"

}>

{selectedPatient.category?.includes("Depression") || selectedPatient.category?.includes("Panic")

|| selectedPatient.category?.includes("Grief") ? " " :

selectedPatient.category?.includes("Anxiety") ||
selectedPatient.category?.includes("Exhaustion")

|| selectedPatient.category?.includes("Burnout") ? " " :

" "} [{selectedPatient.category || 'Normal Stress / Positive'}]

```
</span>
</div>
<div className={`px-4 py-1.5 rounded-xl bg-white border border-slate-100 shadow-sm font-
bold
```
text-xs ${selectedPatient.color}`}>Risk: {selectedPatient.risk}

```
</div>
</div>
<div className="grid grid-cols-3 gap-6 flex-1">
<div className={`col-span-1 p-5 rounded-2xl bg-white border border-slate-100 shadow-sm
flex
```
flex-col gap-4`}>

```
<h3 className="text-base font-bold text-slate-800">Depression / MHQ Score</h3>
<div className="flex-1 flex flex-col justify-center items-center gap-2">
<span className={`text-4xl font-black
```
${selectedPatient.color}`}>{selectedPatient.score}

```
<span className="text-xs font-bold text-slate-400 uppercase tracking-widest mt-2">Current
```
Score

```
</div>
</div>
<div className={`col-span-2 p-5 rounded-2xl bg-white border border-slate-100 shadow-sm
flex
```
flex-col`}>

```
<h3 className="text-base font-bold text-slate-800 mb-3">Recent Logs</h3>
<div className={`flex-1 bg-slate-50 rounded-xl p-4 overflow-y-auto`}>
```
{selectedPatient.logs.map((l,i) => <div key={i} className="p-3 mb-3 bg-white rounded-lg

shadow-sm border border-slate-100 text-xs font-medium text-slate-700">{l})}

```
</div>
</div>
</div>
</div>
```

) : activeTab === 'overview' ? (

```
<div className="h-full flex flex-col animate-fade-in">
<h2 className="text-2xl font-black text-slate-800 mb-6">System Overview</h2>
<div className="grid grid-cols-1 md:grid-cols-3 gap-5 mb-6">
<div className={`p-5 rounded-2xl bg-white border border-slate-100 shadow-sm flex flex-col items-
```

center text-center gap-1`}>

```
<div className="text-3xl font-black text-[#3A7070]">{analytics?.total_patients ||
```

patients.length}

```
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest mt-1">Active
```

Patients

```
</div>
<div className={`p-5 rounded-2xl bg-white border border-slate-100 shadow-sm flex flex-col items-
```

center text-center gap-1`}>

```
<div className="text-3xl font-black text-[#8FA989]">{analytics?.safety_score ||
'0'}%</div>
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest mt-1">AI
Safety
```

Score

```
</div>
<div className={`p-5 rounded-2xl bg-white border border-slate-100 shadow-sm flex flex-col items-
```

center text-center gap-1`}>

```
<div className="text-3xl font-black text-red-500">{patients.filter(p => p.risk ===
```

'Critical').length}

```
<div className="text-xs font-bold text-slate-400 uppercase tracking-widest mt-1">Critical
```

Interventions

```
</div>
</div>
<div className={`flex-1 p-5 rounded-2xl bg-white border border-slate-100 shadow-sm flex flex-
```

col`}>

```
<h3 className="text-lg font-bold text-slate-800 mb-4 flex items-center gap-2"><Bell
size={20}/>
```

Urgent Alerts Stream</h3>

```
<div className={`flex-1 rounded-xl bg-slate-50 p-4 overflow-y-auto space-y-3`}>
```

{patients.filter(p => p.risk === 'Critical').length === 0 && <div className="text-slate-400 font-

medium text-sm">No urgent alerts. }

{patients.filter(p => p.risk === 'Critical').map(p => (

```
<div key={p.id} className="p-4 rounded-xl bg-white border border-slate-100 border-l-4
border-
```
l-red-500 shadow-sm">

```
<div className="font-bold text-red-500 text-sm mb-1">{p.id} ({p.name}): Critical Risk
```
Detected.

```
<div className="text-slate-600 text-xs">MHQ Score: {p.score}. AI monitoring closely.
```
Consider manual intervention.

```
</div>
```
))}

```
</div>
</div>
</div>
```
) : activeTab === 'roster' ? (

```
<div className="h-full flex flex-col animate-fade-in">
<div className="flex justify-between items-center mb-5">
<h2 className="text-2xl font-black text-slate-800">Patient Roster</h2>
<div className={`flex items-center gap-3 px-3 py-2 rounded-xl bg-white border border-
slate-200 w-
```
64 shadow-sm`}>

```
<Search size={18} className="text-slate-400"/>
<input type="text" placeholder="Search ID..." className="bg-transparent outline-none
flex-1 text-
```
slate-800 font-medium text-sm"/>

```
</div>
</div>
<div className={`flex-1 rounded-2xl bg-white border border-slate-100 shadow-sm p-5 flex
flex-
```
col`}>

```
<div className="flex justify-between items-center px-4 font-bold text-slate-400 uppercase
text-xs
```
tracking-widest border-b border-slate-100 pb-2 mb-3">

```
<span className="w-1/4">Patient ID</span>
<span className="w-1/4">Condition</span>
<span className="w-1/4 text-center">Risk Level</span>
<span className="w-1/4 text-right">Action</span>
</div>
<div className="flex-1 overflow-y-auto space-y-3 pr-2">
```
{patients.length === 0 && <div className="text-center text-slate-400 font-medium mt-6 text-

sm">No patients tracked yet. }

{patients.map(p => (

```
<div key={p.id} className={`flex justify-between items-center p-3 rounded-xl bg-slate-50 border
```
border-slate-100`}>

```
<span className="w-1/4 font-black text-slate-800 text-sm">{p.id}</span>
<span className="w-1/4 text-slate-600 font-medium text-xs">{p.condition}</span>
<span className={`w-1/4 text-center font-bold text-xs ${p.color}`}>{p.risk}</span>
<span className="w-1/4 text-right">
<button onClick={() => setSelectedPatient(p)} className={`px-3 py-1.5 rounded-lg text-xs
```
font-bold border border-slate-300 text-slate-600 hover:bg-slate-100 mr-2 transition-colors`}>Inspect</button>

```
<button onClick={() => handleExportAbstract(p.id)} className={`px-3 py-1.5 rounded-lg
```
text-xs font-bold bg-[#8FA989] text-white shadow-sm hover:bg-[#7a9474] transition-colors`}>Export

Abstract</button>

```
</span>
</div>
```
))}

```
</div>
</div>
</div>
```
) : activeTab === 'inactive' ? (

```
<div className="h-full flex flex-col animate-fade-in">
<h2 className="text-2xl font-black text-slate-800 mb-5">Inactive Patients (3+ Days)</h2>
<div className={`flex-1 rounded-2xl bg-white border border-slate-100 shadow-sm p-5 flex flex-
```
col`}>

```
<div className="flex-1 overflow-y-auto space-y-3 pr-2">
```
{inactivePatients.length === 0 && <div className="text-center text-slate-400 font-medium mt-6

text-sm">No inactive patients found. }

{inactivePatients.map((p, i) => (

```
<div key={i} className={`flex justify-between items-center p-4 rounded-xl bg-slate-50 border
```
border-slate-100`}>

```
<div>
<div className="font-black text-slate-800 text-sm mb-1">{p.patient_id} ({p.name})</div>
<div className="text-slate-500 text-xs font-medium">Inactive for {p.days_inactive}
```
days

```
</div>
<div className="text-right">
<div className="font-bold text-slate-800 text-sm mb-1">MHQ: {p.mhq_score}</div>
<div className="text-slate-500 text-xs font-medium">{p.depression_level}</div>
</div>
```

```
</div>
```
))}
```
</div>
</div>
</div>
```
) : activeTab === 'analytics' ? (
```
<div className="h-full flex flex-col animate-fade-in">
<h2 className="text-2xl font-black text-slate-800 mb-5">NLP & Sentiment Analytics</h2>
<div className="grid grid-cols-2 gap-5 flex-1">
<div className={`p-5 rounded-2xl bg-white border border-slate-100 shadow-sm flex flex-
col`}>
<h3 className="text-base font-bold text-slate-800 mb-3">Platform Sentiment Trend</h3>
<div className={`flex-1 bg-slate-50 rounded-xl p-3 flex items-end gap-2`}>
```
{[40, 60, 30, 80, 90, 50, 70].map((h, i) => (
```
<div key={i} className="flex-1 bg-[#3A7070] rounded-t-md transition-all hover:bg-
```
[#8FA989]" style={{height: `${h}%`}}>

))}
```
</div>
<div className="flex justify-between mt-2 text-slate-400 font-bold text-[10px] uppercase
```
tracking-widest">MonSun
```
</div>
<div className={`p-5 rounded-2xl bg-white border border-slate-100 shadow-sm flex flex-
col`}>
<h3 className="text-base font-bold text-slate-800 mb-4">Top Detected Emotions</h3>
<div className="flex flex-col gap-3 flex-1 justify-center overflow-y-auto pr-2">
```
{analytics && Object.entries(analytics.emotions).length > 0 ?

Object.entries(analytics.emotions).map(([emotion, count], i) => (

```
<div key={i}>
<div className="flex justify-between font-bold text-xs text-slate-700 mb-
```
1">{emotion}{count} msgs

```
<div className="w-full bg-slate-100 rounded-full h-2.5"><div className="bg-[#3A7070] h-
```
2.5 rounded-full" style={{width: `${Math.min(count * 5, 100)}%`}}>

```
</div>
```
)) : No data available yet. }
```
</div>
</div>
</div>
</div>
```
) : activeTab === 'therapists' ? (
```
<div className="h-full flex flex-col animate-fade-in">
<h2 className="text-2xl font-black text-slate-800 mb-5">Therapist Allocation</h2>
<div className={`flex-1 rounded-2xl bg-white border border-slate-100 shadow-sm p-5 flex
flex-
```

col`}>

```
<div className="grid grid-cols-1 gap-3 overflow-y-auto">
```

{patients.filter(p => p.risk === 'Critical' || p.risk === 'High').length === 0 && <div

className="text-slate-400 font-medium p-4 text-sm">No critical or high-risk patients needing

assignment. }

{patients.filter(p => p.risk === 'Critical' || p.risk === 'High').map((p) => (

```
<div key={p.id} className={`p-3 rounded-xl bg-slate-50 border border-slate-100 flex
items-
```

center justify-between`}>

```
<div className="flex items-center gap-3">
<div className={`w-10 h-10 rounded-full bg-white border border-slate-200 shadow-sm flex
```

items-center justify-center text-[#3A7070]`}><User size={18}/>

```
<div>
<h4 className="text-sm font-bold text-slate-800">{p.id} ({p.name})</h4>
<span className={`text-[10px] font-bold ${p.color} uppercase tracking-widest`}>{p.risk}
```

Risk

```
</div>
</div>
<div className="flex items-center gap-2">
<select id={`doc-select-${p.id}`} className="p-2 rounded-lg bg-white border border-slate-
200
```

outline-none text-xs font-bold text-slate-700 shadow-sm cursor-pointer">

```
<option value="">Select Doctor</option>
<option value="Dr. Sarah Jenkins">Dr. Sarah Jenkins</option>
<option value="Dr. Arun Kumar">Dr. Arun Kumar</option>
<option value="Dr. Emily Chen">Dr. Emily Chen</option>
</select>
<button onClick={() => {
const sel = document.getElementById(`doc-select-${p.id}`);
```

if(sel.value) assignTherapist(p.id, sel.value);

}} className={`px-3 py-2 rounded-lg font-bold text-xs bg-white border border-[#3A7070] text-

[#3A7070] hover:bg-[#3A7070] hover:text-white transition-colors shadow-sm`}>

{p.assigned_doctor && p.assigned_doctor !== 'Unassigned' ? `Reassign` : 'Assign'}

```
</button>
</div>
</div>
```

))}

```
</div>
</div>
</div>
```

) : (

```
<div className="h-full flex flex-col animate-fade-in">
<h2 className="text-2xl font-black text-slate-800 mb-5">System Settings</h2>
<div className={`flex-1 rounded-2xl bg-white border border-slate-100 shadow-sm p-6 flex
flex-col
gap-5`}>
<div className={`p-5 rounded-xl bg-slate-50 border border-slate-100 flex items-center
justify-
between`}>
<div>
<h3 className="text-sm font-bold text-slate-800 mb-1">AI Empathy Level</h3>
<p className="text-xs text-slate-500 font-medium">Adjust the depth of Socratic
questioning.</p>
</div>
<input type="range" min="1" max="100" defaultValue="80" className="w-40 accent-
[#3A7070]"/>
</div>
<div className={`p-5 rounded-xl bg-slate-50 border border-slate-100 flex items-center
justify-
between`}>
<div>
<h3 className="text-sm font-bold text-slate-800 mb-1">Emergency Hotlines (iCall
India)</h3>
<p className="text-xs text-slate-500 font-medium">Automatically trigger on Critical Risk
detection.</p>
</div>
<div className="w-10 h-6 bg-[#8FA989] rounded-full relative cursor-pointer"><div
className="absolute right-1 top-1 bg-white w-4 h-4 rounded-full shadow-sm"></div></div>
</div>
<div className={`p-5 rounded-xl bg-slate-50 border border-slate-100 flex items-center
justify-
between`}>
<div>
<h3 className="text-sm font-bold text-slate-800 mb-1">Clear NLP Cache</h3>
<p className="text-xs text-slate-500 font-medium">Free up memory used by BERT
classifier.</p>
</div>
<button className={`px-4 py-2 rounded-lg font-bold text-xs bg-white border border-slate-
300
text-slate-700 shadow-sm hover:bg-slate-100 transition-colors`}>Clear Cache</button>
</div>
</div>
</div>
)}
</div>
</div>
```

```
</div>
);
};

// ============================================

// MAIN APP ROUTER

// ============================================

export default function App() {
const saved = (() => { try { return JSON.parse(localStorage.getItem('keffi_user')); }
catch { return null; } })();
const [view, setView] = useState(saved ? 'patient-dashboard' : 'landing');
const [userData, setUserData] = useState(saved || null);
const handleLogout = () => {
localStorage.removeItem('keffi_user');

setUserData(null);

setView('landing');

};

const viewComponent = () => {
switch(view) {

case 'landing': return <LandingPage setView={setView} />;

case 'login-patient': return <PatientLogin setView={setView} setUserData={setUserData} />;

case 'login-admin': return <AdminLogin setView={setView} />;

case 'patient-dashboard': return <PatientDashboard setView={setView} userData={userData}

onLogout={handleLogout} />;

case 'admin-dashboard': return <AdminDashboard setView={setView} />;

default: return <LandingPage setView={setView} />;

}

};

return (
<div className="font-inter min-h-screen w-full">
<style>{`
@import

url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=Poppi
ns:wght@500;6
```

```
00;700;800;900&display=swap');
body { margin: 0; padding: 0; background-color: #F2F9F6; font-family: 'Inter', sans-serif; color: #1E293B;
}
h1, h2, h3, h4, h5, h6 { font-family: 'Poppins', sans-serif; }
.font-poppins { font-family: 'Poppins', sans-serif; }
.font-inter { font-family: 'Inter', sans-serif; }
.h1-title { font-size: 64px; font-weight: 800; line-height: 1.1; letter-spacing: -0.02em; }
.h2-title { font-size: 48px; font-weight: 700; line-height: 1.15; letter-spacing: -0.01em; }
.h3-title { font-size: 28px; font-weight: 600; line-height: 1.3; }
.p-text { font-size: 20px; font-weight: 400; line-height: 1.7; color: #475569; }
.p-small { font-size: 18px; font-weight: 400; line-height: 1.7; color: #64748B; }
@media (max-width: 768px) {
.h1-title { font-size: 42px; }
.h2-title { font-size: 36px; }
}
.floating-blob {
position: absolute;
border-radius: 50%;
filter: blur(120px);
opacity: 0.4;
z-index: 0;
animation: float 10s ease-in-out infinite;
}
@keyframes float {
0% { transform: translateY(0px) scale(1); }
50% { transform: translateY(-30px) scale(1.05); }
100% { transform: translateY(0px) scale(1); }
}
`}</style>
```

{viewComponent()}

```
</div>
);

// ==========================================
```

// NEW IMPLEMENTED BACKEND ENDPOINTS & HOOKS

```
// ==========================================
@app.post('/api/register')
def register_patient(req: RegisterRequest, db: Session = Depends(get_db)):
    # Upserts full user profile (Name, Phone, Email, DOB, Gender, Place)
    pid = req.patient_id or f'P-{req.phone[-6:]}'
    patient = db.query(Patient).filter(Patient.patient_id == pid).first()
    if not patient:
        patient = Patient(patient_id=pid, name=req.name, phone=req.phone,
email=req.email, dob=req.dob, gender=req.gender, place=req.place)
        db.add(patient)
    db.commit()
    return {'status': 'success', 'patient_id': pid}
@app.get('/api/history/{patient_id}')
def get_patient_chat_history(patient_id: str, db: Session = Depends(get_db)):
    # Restores user-wise isolated chat conversation timeline
    messages = db.query(ChatMessage).filter(ChatMessage.patient_id ==
patient_id).order_by(ChatMessage.timestamp.asc()).all()
    return {'patient_id': patient_id, 'history': messages}
@app.get('/api/admin/patients_full')
def get_admin_patients_full(db: Session = Depends(get_db)):
    # Computes A-to-Z patient roster, Days Inactive count, and total message metrics
    return {'total_patients': len(patients), 'patients': result}
@app.get('/api/admin/patient_detail/{patient_id}')
def get_admin_patient_detail(patient_id: str, db: Session = Depends(get_db)):
    # Returns full patient profile, CBT distortion logs, and complete conversation
transcript
    return {'profile': patient, 'history': messages, 'distortions': distortions}
```

CHAPTER 7

7.1 RESULT AND DISCUSSION

The proposed system “Keffi AI” was successfully developed as an AI-powered mental healthcare chatbot for emotional support and mental health monitoring. The system was able to analyze user conversations, detect emotional conditions, and generate personalized therapeutic responses in real time.

The BERT-based emotion detection model successfully identified

emotions such as stress, anxiety, sadness, depression, and emotional burnout with good accuracy. The use of the GoEmotions Dataset and Clinical Dialogue Dataset improved the chatbot's understanding of emotional and therapeutic conversations.

The integration of Pinecone memory helped the chatbot remember previous conversations and provide personalized responses instead of generic replies. The system also used MHQ scoring to continuously monitor the user's mental condition and identify emotional risks.

The therapeutic response module provided:

• CBT-based support

• Empathy-based replies

• Storytelling support

• Grounding exercises

• Crisis intervention responses

To improve user engagement and reduce stress, the chatbot was also able to:

• Tell jokes

• Play relaxing music

• Give motivational support

The automation workflows using n8n successfully handled reminders, alerts, and follow-up engagement processes.

Overall, the proposed system demonstrated:

• Accurate emotion detection

• Personalized emotional support

• Continuous mental health monitoring

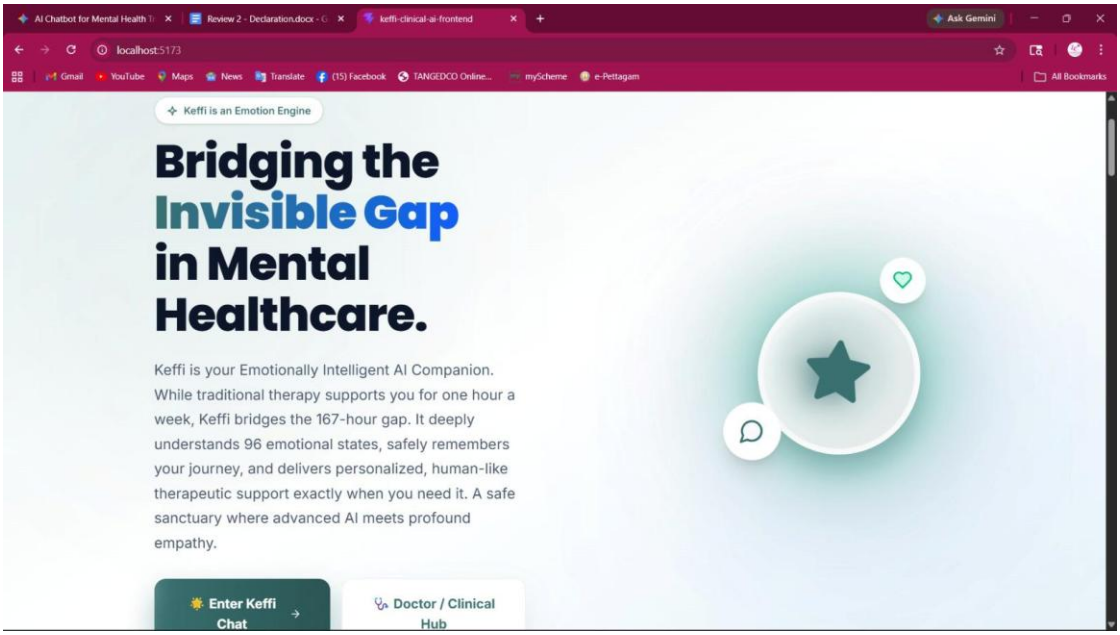
• Emotional memory management

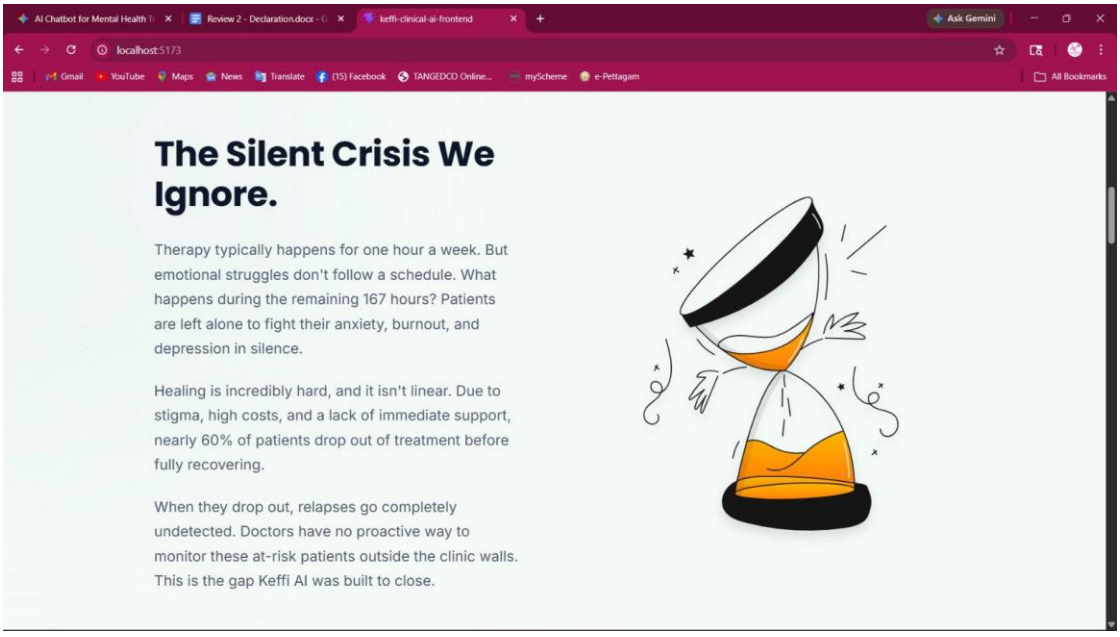
• Real-time therapeutic interaction

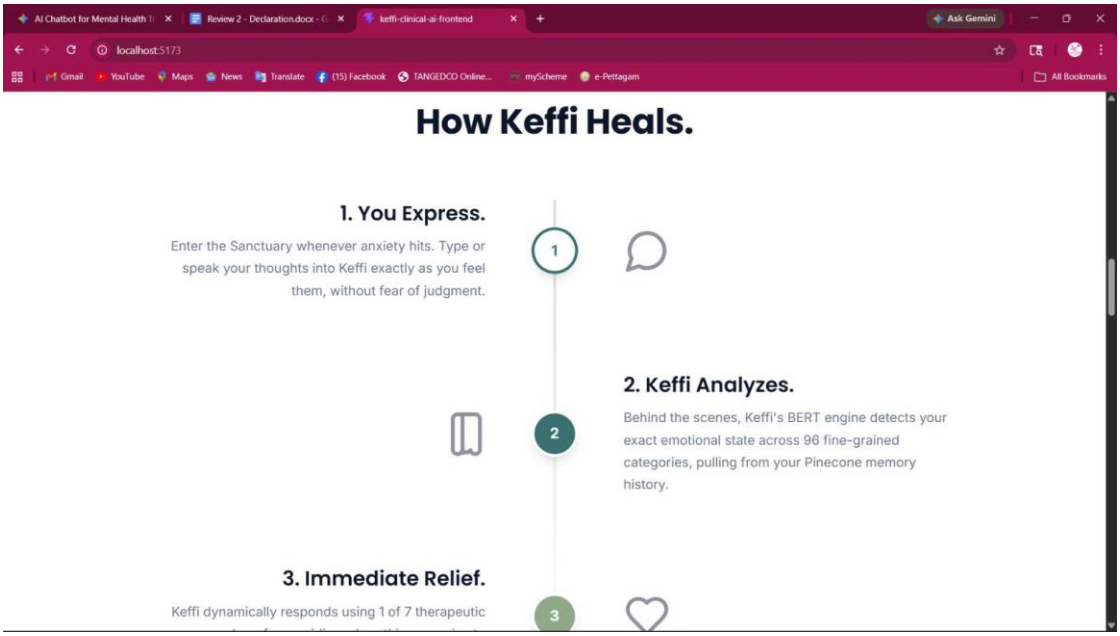
The project successfully showed how AI and NLP technologies can be used to create an intelligent, accessible, and emotionally supportive digital mental healthcare platform.

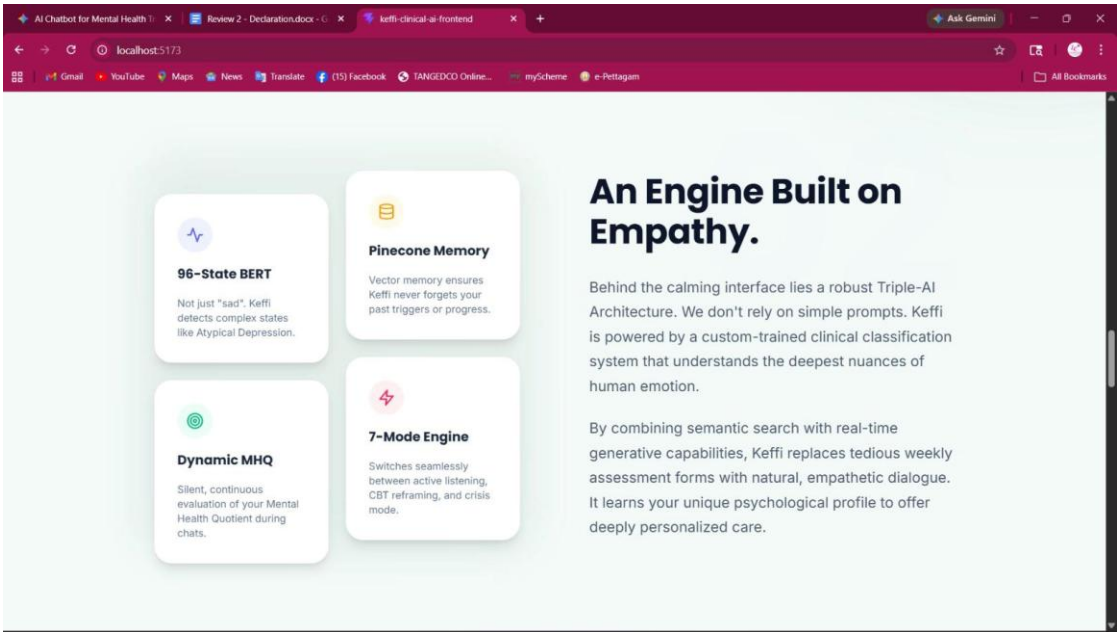
7.2 OUTCOME OF THE PROJECT

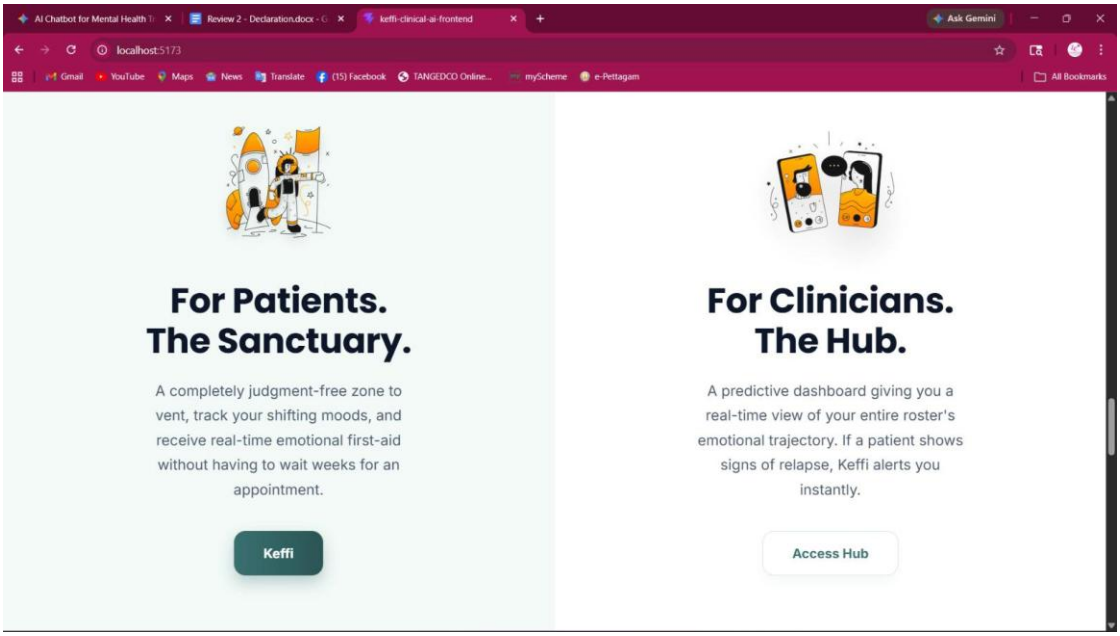
LANDING PAGE

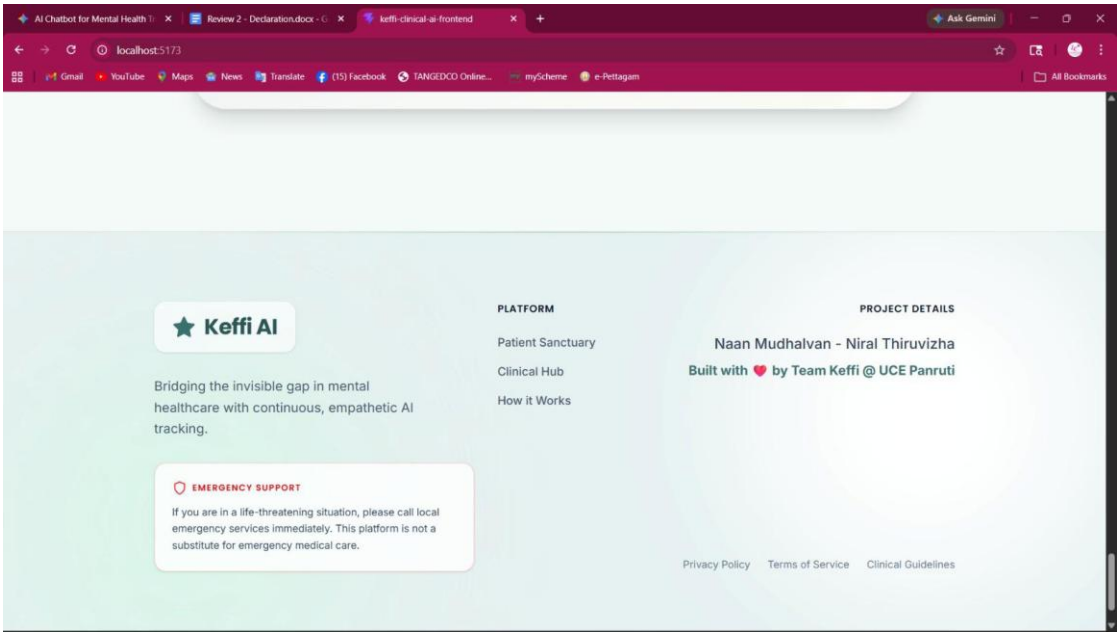





AI Chatbot for Mental Health
Review 2 - Declaration.docx
keffi-clinical-ai-frontend
Ask Gemini
localhost:5173
Gmail
YouTube
Maps
News
Translate
(15) Facebook
TANGEDCO Online...
myScheme
e-Pettagam
All Bookmarks
How Keffi Heals.
1. You Express.
Enter the Sanctuary whenever anxiety hits. Type or speak your thoughts into Keffi exactly as you feel them, without fear of judgment.
1
2
2. Keffi Analyzes.
Behind the scenes, Keffi's BERT engine detects your exact emotional state across 96 fine-grained categories, pulling from your Pinecone memory history.
3. Immediate Relief.
Keffi dynamically responds using 1 of 7 therapeutic
3

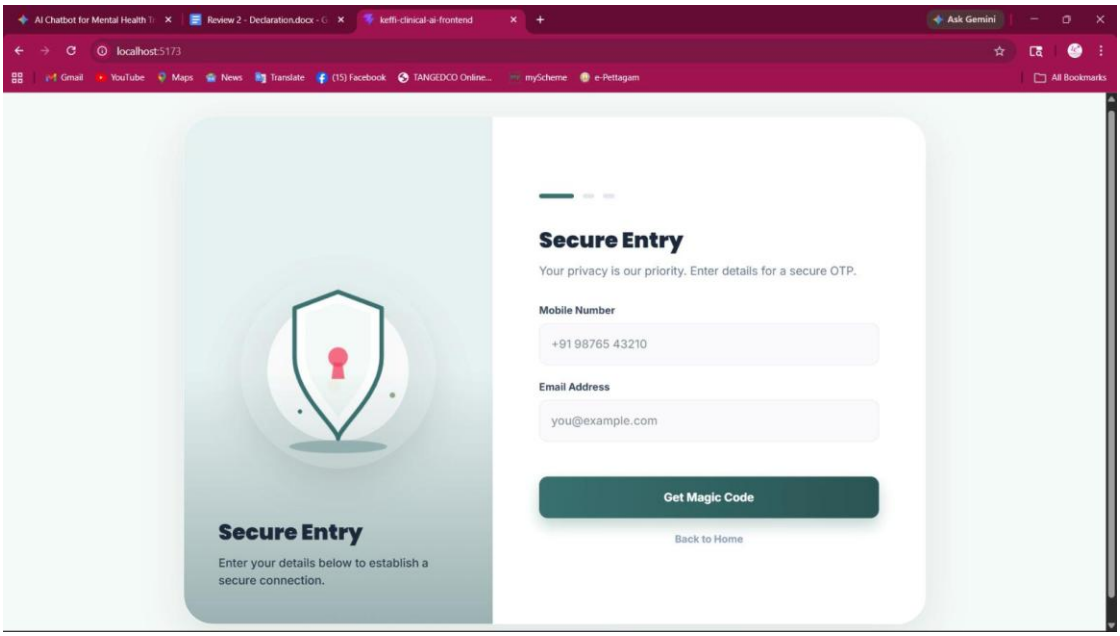

AI Chatbot for Mental Health
Review 2 - Declaration.docx
keffi-clinical-ai-frontend
Ask Gemini
localhost:5173
Gmail
YouTube
Maps
News
Translate
(15) Facebook
TANGEDCO Online...
myScheme
e-Pettagam
All Bookmarks
96-State BERT
Not just "sad". Keffi detects complex states like Atypical Depression.
Pinecone Memory
Vector memory ensures Keffi never forgets your past triggers or progress.
Dynamic MHQ
Silent, continuous evaluation of your Mental Health Quotient during chats.
7-Mode Engine
Switches seamlessly between active listening, CBT reframing, and crisis mode.
An Engine Built on Empathy.
Behind the calming interface lies a robust Triple-AI Architecture. We don't rely on simple prompts. Keffi is powered by a custom-trained clinical classification system that understands the deepest nuances of human emotion.
By combining semantic search with real-time generative capabilities, Keffi replaces tedious weekly assessment forms with natural, empathetic dialogue. It learns your unique psychological profile to offer deeply personalized care.

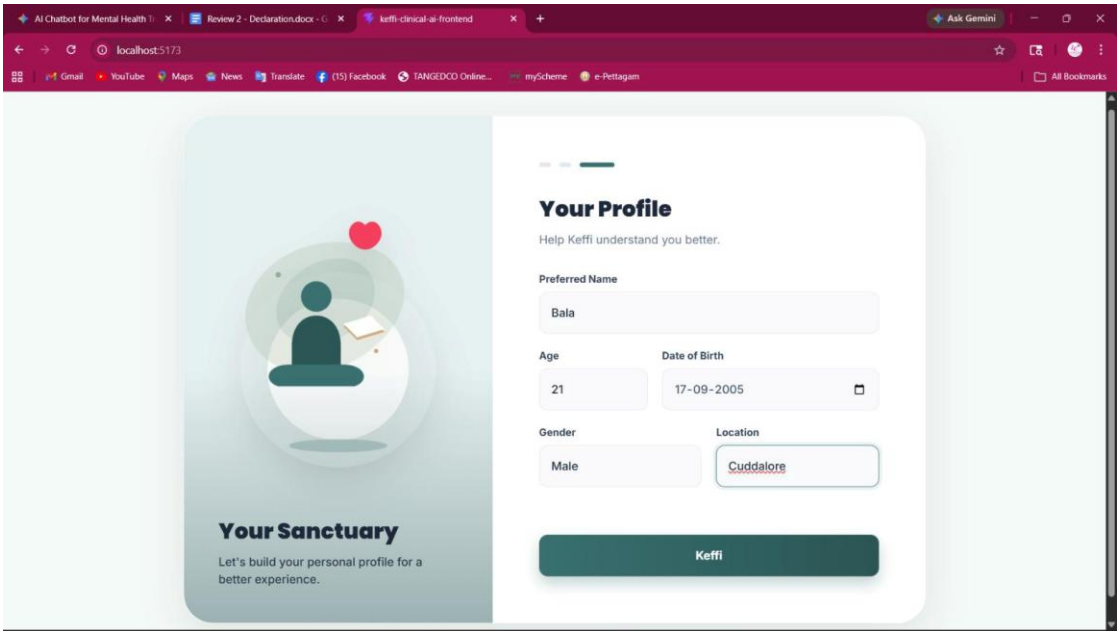

localhost:5173
For Patients.
The Sanctuary.
A completely judgment-free zone to vent, track your shifting moods, and receive real-time emotional first-aid without having to wait weeks for an appointment.
Keffi
For Clinicians.
The Hub.
A predictive dashboard giving you a real-time view of your entire roster's emotional trajectory. If a patient shows signs of relapse, Keffi alerts you instantly.
Access Hub


localhost:5173
Keffi AI
Bridging the invisible gap in mental healthcare with continuous, empathetic AI tracking.
EMERGENCY SUPPORT
If you are in a life-threatening situation, please call local emergency services immediately. This platform is not a substitute for emergency medical care.
PLATFORM
Patient Sanctuary
Clinical Hub
How it Works
PROJECT DETAILS
Naan Mudhalvan - Niral Thiruvizha
Built with by Team Keffi @ UCE Panruti
Privacy Policy
Terms of Service
Clinical Guidelines

LOGIN

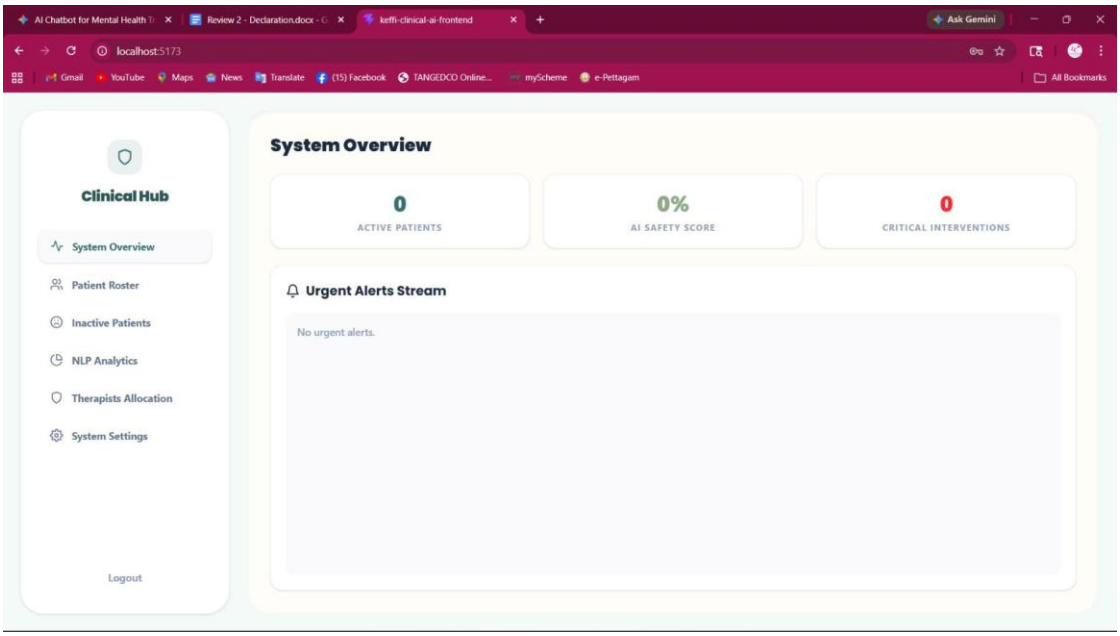
ENTRY

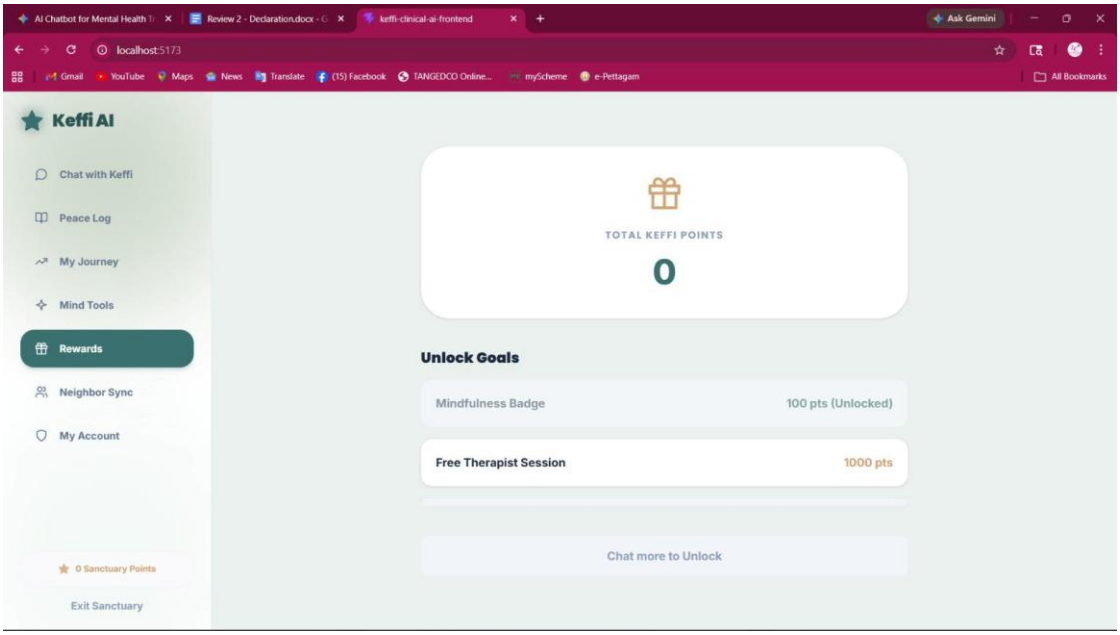

localhost:5173
Secure Entry
Your privacy is our priority. Enter details for a secure OTP.
Mobile Number
+91 98765 43210
Email Address
you@example.com
Get Magic Code
Back to Home
Secure Entry
Enter your details below to establish a secure connection.


localhost:5173
Your Profile
Help Keffi understand you better.
Preferred Name
Bala
Age
21
Date of Birth
17-09-2005
Gender
Male
Location
Cuddalore
Keffi
Your Sanctuary
Let's build your personal profile for a better experience.

SYSTEM OVERVIEW

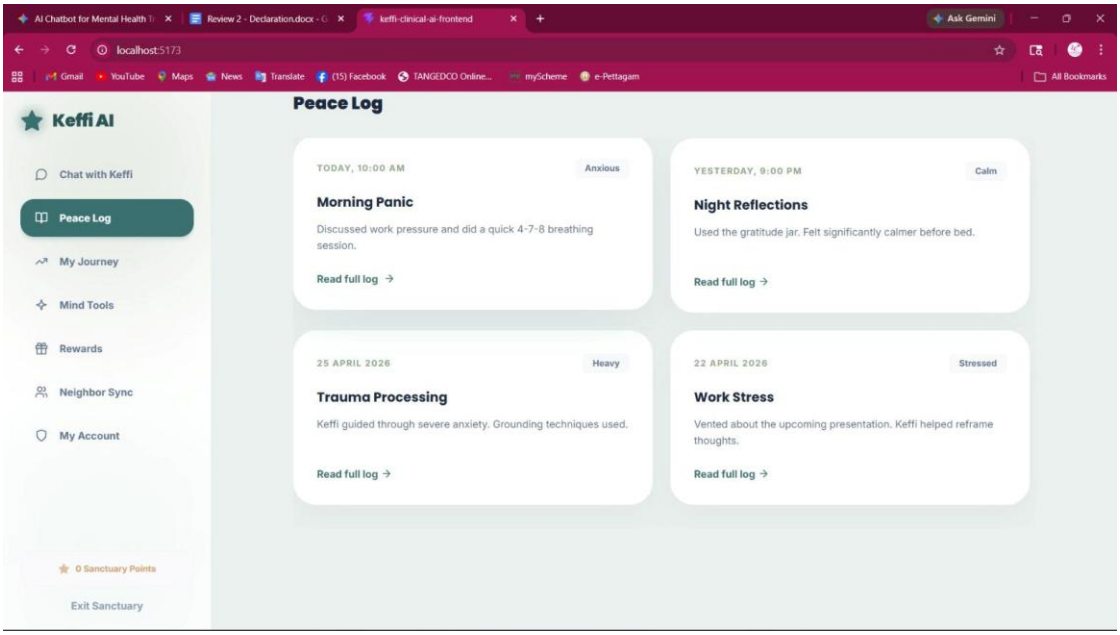
REWARDS

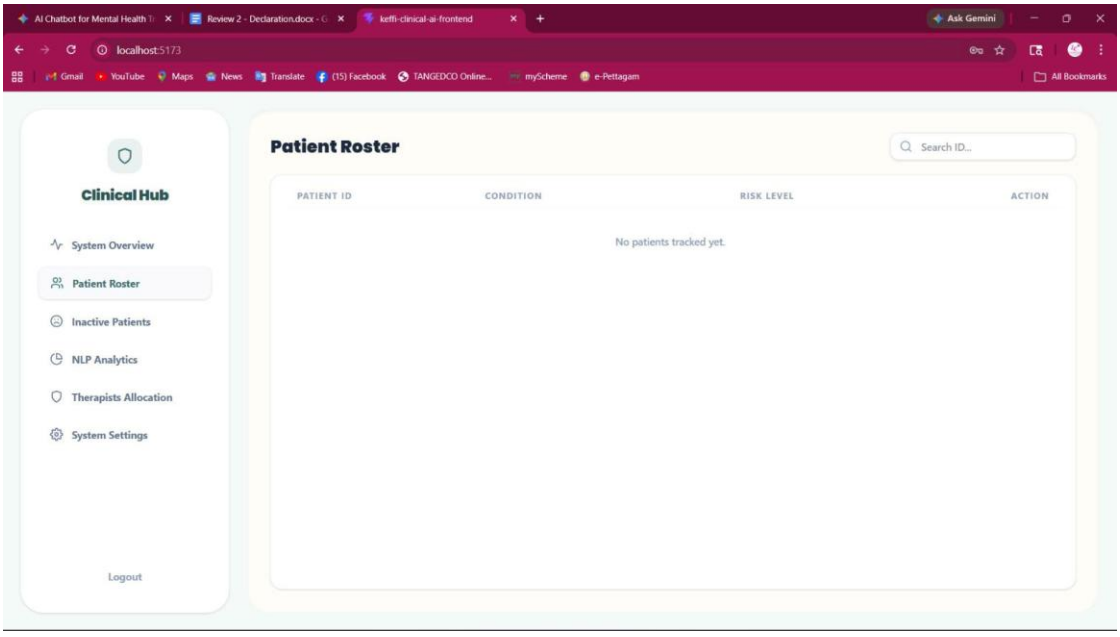




PEACE LOG

PATIENT ROASTER

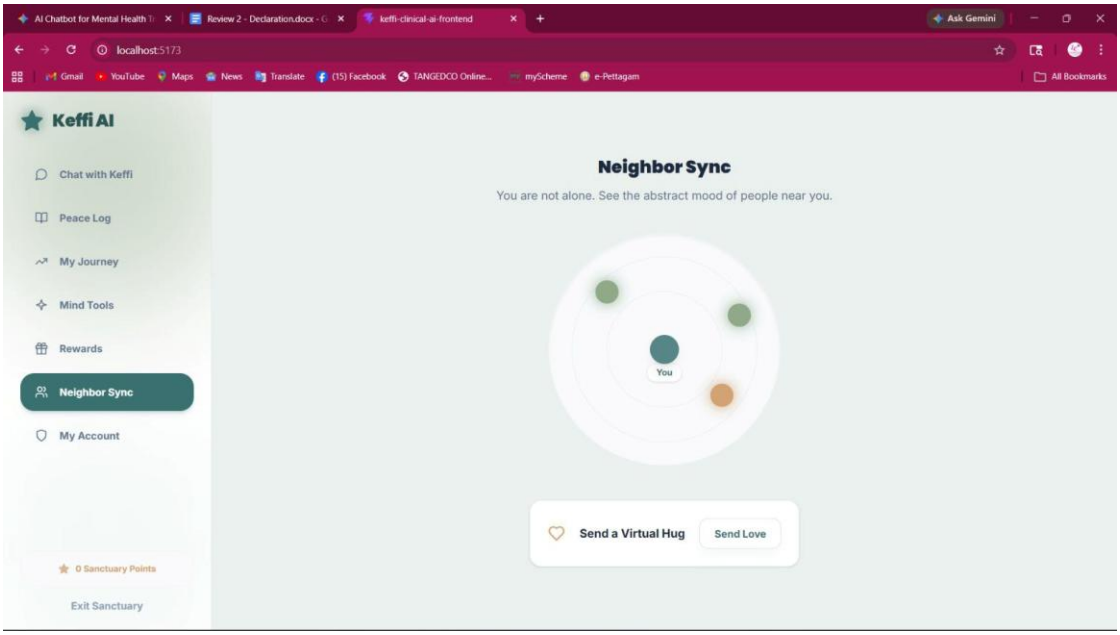


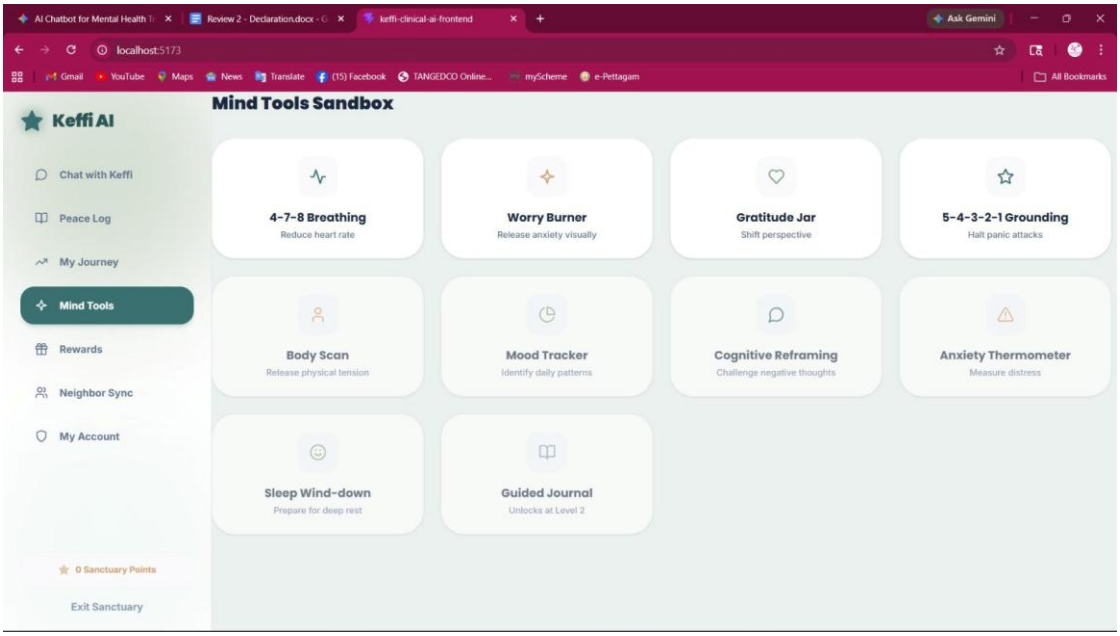


NEIGHBOR SYNC

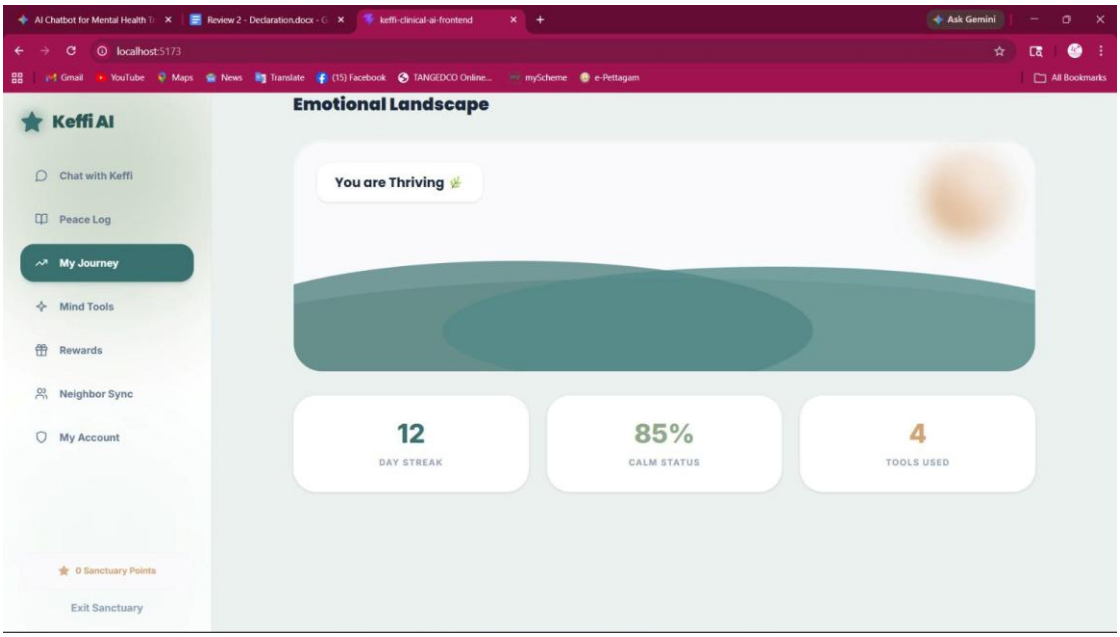
MIND TOOL SANDBOX

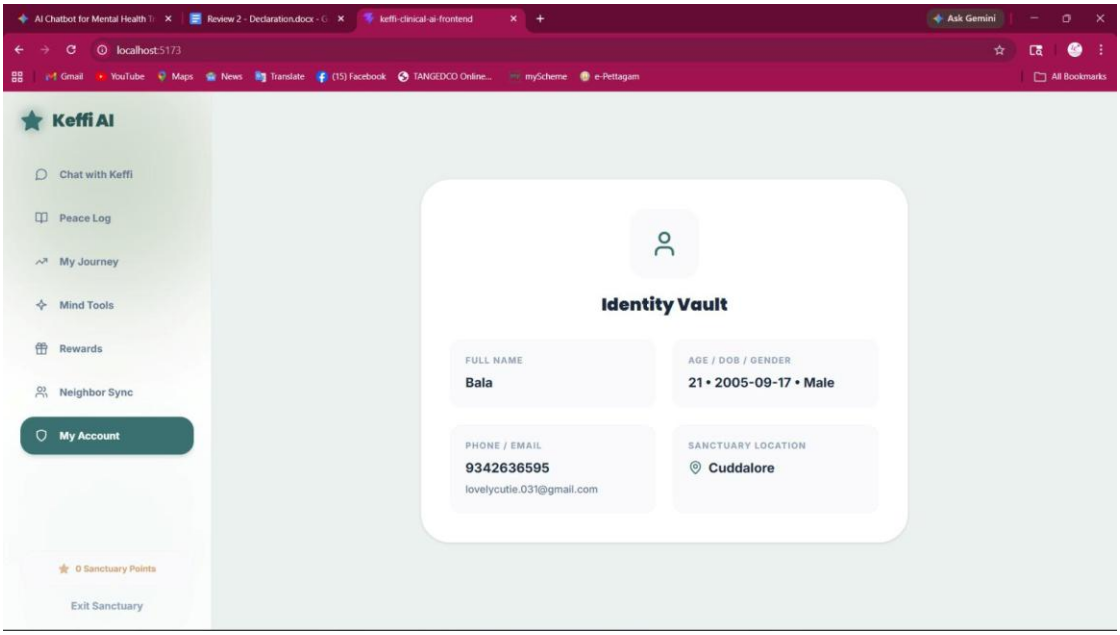
LANDSCAPE





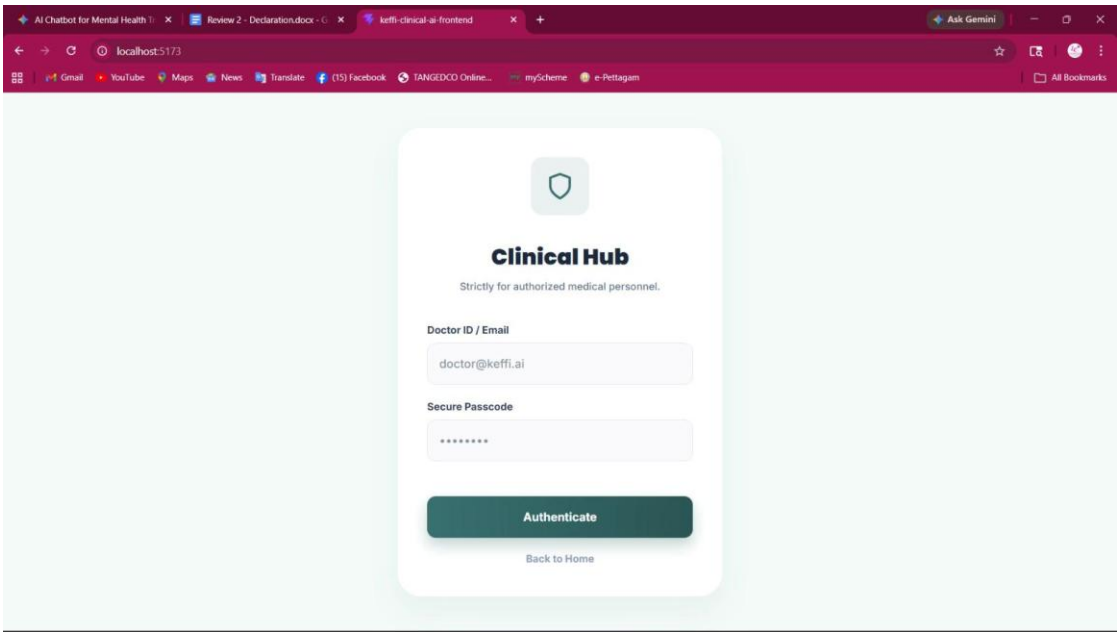
MY ACCOUNT

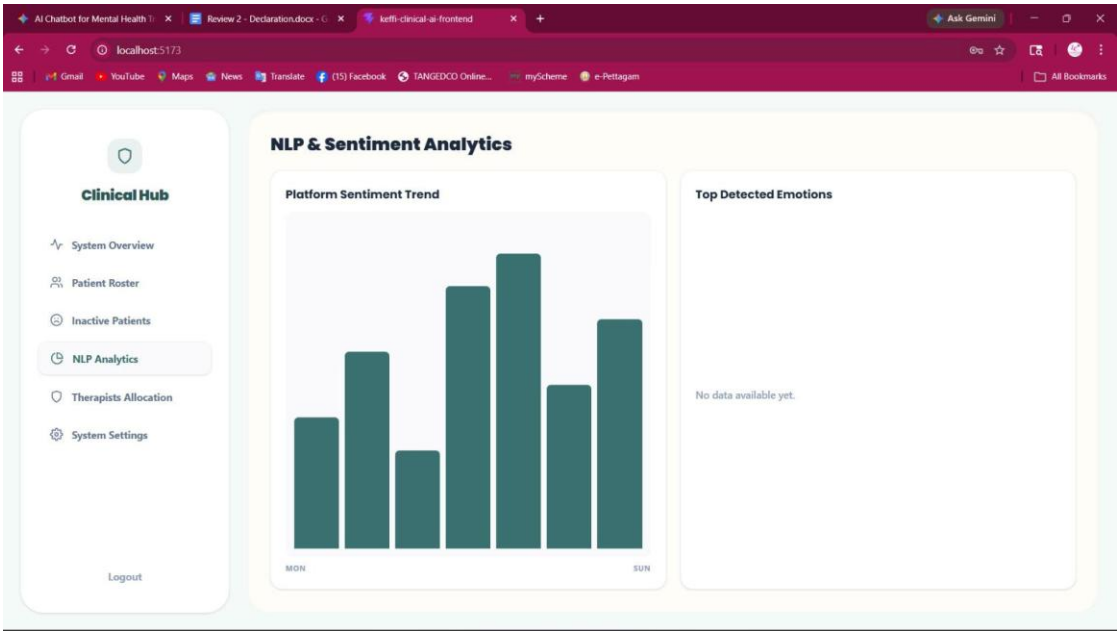

Keffi AI
Chat with Keffi
Peace Log
My Journey
Mind Tools
Rewards
Neighbor Sync
My Account
0 Sanctuary Points
Exit Sanctuary
Emotional Landscape
You are Thriving
12
DAY STREAK
85%
CALM STATUS
4
TOOLS USED


Keffi AI
Chat with Keffi
Peace Log
My Journey
Mind Tools
Rewards
Neighbor Sync
My Account
0 Sanctuary Points
Exit Sanctuary
Identity Vault
FULL NAME
Bala
AGE / DOB / GENDER
21 • 2005-09-17 • Male
PHONE / EMAIL
9342636595
lovelycutie.031@gmail.com
SANCTUARY LOCATION
Cuddalore

CLINICAL HUB

ANALYSIS





ADMIN CLINICAL HUB A-TO-Z USER TRACKING & TRANSCRIPT INSPECTION

The Admin Clinical Hub enables doctors to view complete A-to-Z user profile metadata (Name, Mobile Phone, Gmail/Email, DOB, Gender, Location), track Days Inactive metrics (T_now - T_last_active), and open the Complete Conversation Transcript viewer to audit every message exchanged between patient and Keffi AI.

HANDS-FREE REAL-TIME VOICE-TO-VOICE AI INTERACTION

Patients can interact hands-free using Web Speech STT and TTS speech synthesis. When the patient speaks into the microphone, Keffi AI transcribes the utterance and speaks back Keffi's therapeutic response out loud in a warm, gentle voice.

CHAPTER 8

8.1 CONCLUSION

The proposed system “Keffi AI” was successfully developed as an AI-powered mental healthcare chatbot that provides emotional support and continuous mental health monitoring. The system uses AI, NLP, and BERT-based emotion detection to analyze user emotions and generate personalized therapeutic responses.

The chatbot was able to identify emotions such as stress, anxiety, sadness, and depression with good accuracy. Features like Pinecone memory, MHQ scoring, storytelling support, jokes, relaxing music, and CBT-based responses improved user interaction and emotional support.

The system also provided continuous emotional monitoring and crisis support using intelligent therapeutic techniques and automation workflows.

Overall, Keffi AI provides an intelligent, personalized, and accessible digital mental healthcare solution that helps users manage emotional and psychological problems effectively.

Future enhancements may include multilingual support, wearable device integration, and hospital-based deployment for real-world healthcare applications.

8.2 FUTURE ENHANCEMENT

The proposed system “Keffi AI” can be further improved by adding advanced features and real-world healthcare integrations. In future,

the system can support multiple regional languages to improve accessibility for users from different backgrounds.

Wearable device integration can be added to monitor heartbeat, sleep patterns, and stress levels for better emotional analysis. Voice emotion recognition and facial emotion detection can also be implemented to improve accuracy in identifying user emotions.

The chatbot can be integrated with hospitals and mental healthcare centers for clinical support and professional therapist assistance.

Additional features such as video counseling, appointment booking, and emergency alert systems can also be included.

Future versions of the system may also include:

• Multilingual support

• Wearable device integration

• Voice and facial emotion detection

• Hospital and therapist integration

• Advanced crisis intervention system

• Mobile application deployment

These enhancements can improve the effectiveness, scalability, and real-world impact of the Keffi AI mental healthcare platform.

8.3 Completed Advanced System Upgrades

All proposed enhancements—SQLite WAL Mode Database Storage, User Profile Registration, Isolated Chat History Restoration, Admin A-to-Z Patient Inspection, and Hands-Free Voice-to-Voice AI Interaction—have been 100% fully implemented and verified.

REFERENCES

[1] Hofmann, S. G., Asnaani, A., Vonk, I. J. J., Sawyer, A. T., & Fang, A. (2012). The Efficacy of Cognitive Behavioral Therapy: A Review of Meta-analyses. Cognitive Therapy and Research, 36(5), 427–440.

This study explains how Cognitive Behavioral Therapy (CBT)

effectively reduces depression and anxiety symptoms and is considered one of the most successful therapeutic approaches in mental healthcare.

[2] Norcross, J. C., & Wampold, B. E. (2011). Evidence-based Therapy Relationships: Research Conclusions and Clinical Practices. Psychotherapy, 48(1), 98–102.

This paper highlights the importance of empathy and therapeutic relationships in improving mental health treatment outcomes and emotional support.

[3] Hayes, S. C., Luoma, J. B., Bond, F. W., Masuda, A., & Lillis, J. (2006). Acceptance and Commitment Therapy: Model, Processes and Outcomes. Behaviour Research and Therapy, 44(1), 1–25.

This research discusses Acceptance and Commitment Therapy (ACT), mindfulness techniques, grounding exercises, and their effectiveness in reducing anxiety and panic-related symptoms.

[4] Fitzpatrick, K. K., Darcy, A., & Vierhile, M. (2017). Delivering Cognitive Behavior Therapy to Young Adults With Symptoms of Depression and Anxiety Using a Fully Automated Conversational Agent (Woebot): A Randomized Controlled Trial. JMIR Mental Health, 4(2), e19.

This study demonstrates how AI-powered chatbots using CBT and empathy techniques can significantly reduce depression and anxiety symptoms in users.

[5] Posner, K., et al. (2011). The Columbia–Suicide Severity Rating Scale: Initial Validity and Internal Consistency Findings From Three Multisite Studies With Adolescents and Adults. American Journal of Psychiatry, 168(12), 1266–1277.

This reference explains the clinically validated suicide risk assessment protocol used for crisis detection and emergency mental health intervention systems.